Tribhuvan University

Institute of Science and Technology

A Project Report On

"Mindless System(Customer Intelligence System)"

Submitted to:

Department of Computer Science and Information Technology

Patan Multiple Campus

Patandhoka, Lalitpur

In partial fulfillment of the requirements

For the Bachelors of Science in Computer Science and Information Technology

Submitted by:

Adarsha Joshi(79010007)

Apil Paudel(79010018)

Semester: VII

July 20, 2026

Under the supervision of:

Mr.Bikash Balami

Tribhuvan University

Institute of Science and Technology

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project report prepared under my supervision by Adarsha Joshi and Apil Paudel entitled "**Mindless System (Customer Intelligence Engine)**" in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Information Technology to be processed for final evaluation.

……

Mr. Bikash Balami

Supervisor

Department of Statistics and Computer Science

Patan Multiple Campus, Lalitpur

Tribhuvan University

Institute of Science and Technology

CERTIFICATE OF APPROVAL

This is to certify that this project prepared by Adarsha Joshi and Apil Paudel entitled "Mindless System (Customer Intelligence Engine)" in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Information Technology has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

……	……
Mr. Bikash Balami	[Internal Examiner]
Supervisor	Internal Examiner
……	……
[Head of Department]	[External Examiner]
Head of Department	External Examiner

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the Department of Statistics and Computer Science, Patan Multiple Campus, for providing us the opportunity to undertake this final year project as part of the Bachelor of Science in Computer Science and Information Technology program.

We are deeply thankful to our supervisor, Mr. Bikash Balami, for continuous guidance, constructive feedback, and encouragement throughout the design and development of this system. We also extend our appreciation to all the faculty members whose teaching over the course of the program built the foundation on which this work stands.

Finally, we thank our families and friends for their patience and support during the completion of this work.

With Respect,

Adarsha Joshi

Apil Paudel

ABSTRACT

Modern online retailers accumulate large volumes of behavioral data, yet conventional analytics tools mostly report surface-level counts and rarely translate raw activity into an actionable understanding of who each customer is and what they are worth. This project addresses that gap by building "Mindless System (Customer Intelligence Engine)", implemented as a working e-commerce platform named ThreadHouse together with an administrative intelligence dashboard.

The system is a full-stack application comprising two React single-page applications (a customer shop and an admin panel) served by a FastAPI backend over a single PostgreSQL database. Every meaningful interaction in the shop—page views, clicks, searches, wishlist and cart actions, checkout starts, and purchases—is captured as an analytics event tagged with user identity and, where relevant, monetary value. Administrators can watch these events arrive in real time over a WebSocket stream and can run a machine-learning pipeline directly on the live order data. The panel also surfaces storefront engagement, highlighting the most-visited pages and the most-wishlisted products.

The intelligence pipeline performs Recency–Frequency–Monetary (RFM) feature extraction and quintile scoring, assigns each customer to one of ten behavioral segments (plus an Anomalous class) using an interpretable rule model, predicts twelve-month Customer Lifetime Value with the BG/NBD and Gamma-Gamma probabilistic models, estimates high-value-repeater potential with a gradient-boosting classifier, and flags anomalous customers using a PyTorch autoencoder with a statistical fallback. A large language model narrates the numerical findings into readable executive insights and answers natural-language questions about the analyzed dataset.

The delivered system demonstrates that behavioral tracking, interpretable segmentation, and predictive modeling can be integrated into a single coherent application, and it is evaluated through unit, integration, and functional testing.

Keywords: Customer Intelligence, RFM Segmentation, Customer Lifetime Value, Anomaly Detection, Real-Time Analytics

TABLE OF CONTENTS

LIST OF ABBREVIATIONS

API	Application Programming Interface
BG/NBD	Beta-Geometric / Negative Binomial Distribution
CLV	Customer Lifetime Value
CRISP-DM	Cross-Industry Standard Process for Data Mining
CSV	Comma-Separated Values
HVR	High-Value Repeater
JWT	JSON Web Token
LLM	Large Language Model
ML	Machine Learning
ORM	Object–Relational Mapping
RFM	Recency, Frequency, Monetary
SPA	Single-Page Application
SQL	Structured Query Language
UML	Unified Modeling Language

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1: INTRODUCTION

1.1 Introduction

Electronic commerce has become one of the most data-rich domains in software. Each visit a customer makes to an online store produces a trail of interactions: pages viewed, products searched, items added to a cart or a wishlist, orders placed or abandoned. Individually these interactions are small, but in aggregate they encode who a customer is, how engaged they are, and how much they are likely to be worth to the business over time.

Despite the abundance of this data, most small and mid-sized retailers rely on descriptive analytics tools that count activity rather than interpret it. This project, titled "Mindless System (Customer Intelligence Engine)", is realized as a complete, working e-commerce platform coupled with an administrative intelligence engine. The shop generates genuine behavioral data, and the intelligence engine transforms that data into customer segments, value predictions, anomaly flags, and plain-language insights. Building both halves within one application allows the analytics to operate on real, first-party data.

1.2 Problem Statement

Although organizations collect large amounts of user data, converting that information into meaningful insight remains difficult. Conventional analytics offer numerical summaries but lack the depth to reveal engagement drivers, value, or shifts in intent. Manual segmentation compounds the limitation because it depends on static rules and predefined assumptions that do not adapt as behavior evolves. In practice this produces three concrete problems that this project set out to solve:

- Behavioral data is captured (if at all) in a form that is disconnected from customer identity and monetary value, making it hard to reason about individual customers.
- Segmentation, value estimation, and anomaly detection are treated as separate offline exercises rather than as an integrated, repeatable pipeline that an operator can run on demand.
- The output of analytical models is numerical and hard for non-technical staff to act on, so insight rarely reaches the people who make merchandising and retention decisions.

1.3 Objectives

The main objective of the project is to design and develop an integrated customer intelligence engine, embedded within a working e-commerce platform, that transforms real behavioral data into actionable customer understanding. The specific objectives are:

1. To build an e-commerce shop that captures user behavior (including page views, clicks, searches, wishlist and cart actions, checkout starts, and purchases) as identity- and value-tagged analytics events.
2. To segment customers into interpretable behavioral groups using Recency–Frequency–Monetary (RFM) analysis, and to predict customer lifetime value and high-value-repeater potential.
3. To detect anomalous customer behavior and to present all findings, together with large-language-model-generated narrative insights, through an interactive administrative dashboard that also supports real-time monitoring.

1.4 Scope and Limitations

The scope of the delivered system covers a customer-facing shop (registration and login, product browsing by category and search, product detail pages, a cart, a wishlist, and Cash-on-Delivery order placement); an administrative panel (product management with image upload, order management with status updates and an audit trail, real-time live tracking, an engagement view of the most-visited pages and most-wishlisted products, and an intelligence workspace); an intelligence pipeline (RFM segments, CLV, HVR potential, anomaly classification, and narrative insights over an uploaded CSV or the platform's live order data); and a natural-language query facility grounded in the computed results.

The system is a coursework and portfolio project and is not production-hardened. Its principal limitations are:

- Payment is limited to Cash on Delivery; no online payment gateway is integrated.
- Authentication uses stateless JSON Web Tokens with no server-side session revocation beyond token expiry.
- The predictive models depend on data volume: the high-value-repeater classifier runs only when at least 100 customers are present, and lifetime-value estimation requires at least 50 repeat purchasers; below these thresholds the corresponding fields are null.

- The large-language-model features require an external API key; without it, the system relies on deterministic rule-based insights.

1.5 Development Methodology

The project followed an Agile software development methodology [5], delivered through short iterative and incremental cycles, complemented by the CRISP-DM (Cross-Industry Standard Process for Data Mining) process [4] for the analytical component. Agile was chosen because the system is naturally decomposable into independent increments (the shop, the behavioral-tracking layer, the real-time dashboard, and each analytical stage) that could be planned, built, tested, and reviewed in short sprints, with each sprint producing working, demonstrable functionality on a stable base. This suited a small two-member team and an academic timeline, accommodated changing requirements as the working system revealed what was practical, and matched the exploratory nature of the machine-learning work, where a stage was only accepted into an increment once it produced correct results on sample data.

Within each Agile iteration the standard activities of requirement refinement, design, implementation, and testing were carried out. The analytical iterations were additionally guided by the six phases of CRISP-DM, which provided a disciplined data-mining sub-process inside the wider Agile flow:

- **Business understanding:** defining the goal of turning raw behavior into actionable customer understanding.
- **Data understanding:** exploring transaction and event data (recency, frequency, monetary value, tenure, basket composition).
- **Data preparation:** schema detection, cleaning, feature extraction, and robust scaling.
- **Modeling:** RFM scoring and segmentation, BG/NBD and Gamma-Gamma lifetime-value estimation, gradient-boosting HVR prediction, and autoencoder anomaly detection.
- **Evaluation:** validating outputs on sample and live data and confirming graceful degradation on small samples.
- **Deployment:** integrating the pipeline behind the admin dashboard with narrative insight generation.

The combination gave the project the flexibility and rapid feedback of Agile for the application as a whole, together with the rigor of a recognized data-mining process for the intelligence engine at its core.

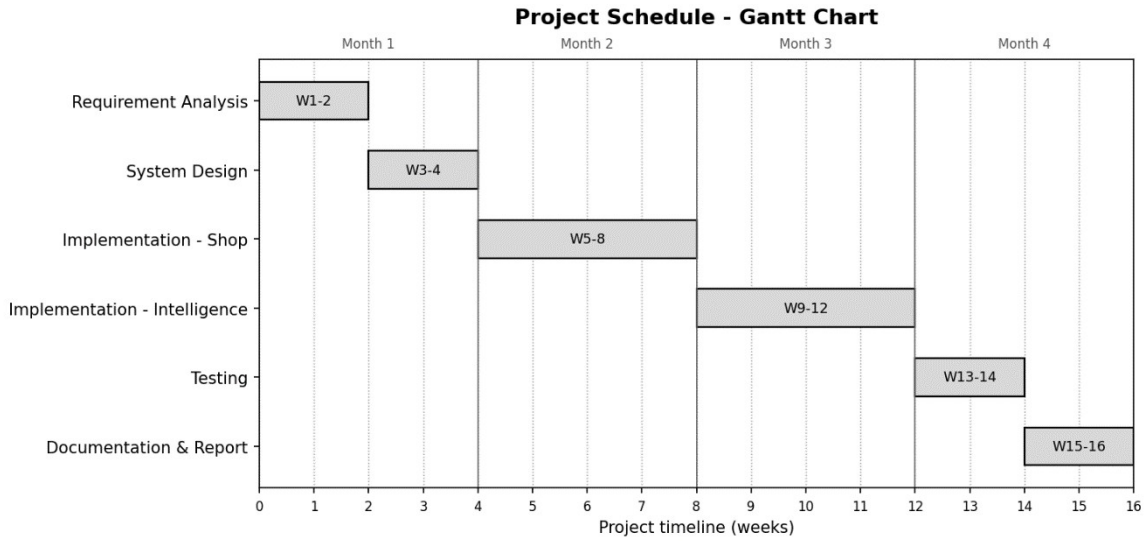


Figure 1.1:Gantt Chart

1.6 Report Organization

The remainder of this report is organized as follows. Chapter 2 presents the background study and literature review. Chapter 3 covers system analysis, including requirement analysis, feasibility, and the object-oriented analysis models. Chapter 4 presents the system design, the refined structural and behavioral models, the component and deployment views, the database design, and the algorithm details. Chapter 5 describes the implementation, the user interface, and the testing. Chapter 6 concludes with the conclusion and future recommendations, followed by references and appendices.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

Customer intelligence is the practice of turning behavioral and transactional data into an understanding of customer value and intent. Three techniques underpin this project. Recency–Frequency–Monetary (RFM) analysis characterizes each customer by how recently they purchased, how often, and how much they spent, and remains popular because it is interpretable and inexpensive to compute [7]. Customer Lifetime Value (CLV) modeling estimates the future worth of a customer; for non-contractual retail settings the probabilistic "buy-till-you-die" family (the BG/NBD model for transaction count paired with the Gamma-Gamma model for monetary value) is a standard approach [8]. Anomaly detection identifies customers whose behavior deviates from the norm; autoencoders, a class of unsupervised neural networks that learn to reconstruct their input, are widely used for this purpose through their reconstruction error [6].

These techniques operate on behavioral data that must first be captured. In an e-commerce context this means instrumenting the storefront so that page views, clicks, searches, and cart, wishlist, and purchase actions are recorded and associated with a customer and, where applicable, a monetary value, the foundation on which the analytical layer of this project is built.

2.2 Literature Review

2.2.1 Existing Systems

Established web-analytics platforms such as Google Analytics [1] and Mixpanel [2] focus on descriptive reporting: they aggregate visits, events, funnels, and conversion rates. These tools are effective for measuring activity, but they are built around predefined metrics and offer limited native support for per-customer value modeling or unsupervised behavioral discovery. Customer-relationship and customer-data platforms provide richer profiles, but they are commercial, general-purpose, and typically decoupled from a specific store’s transactional model.

2.2.2 Related Technologies

- **FastAPI.** A modern asynchronous Python web framework providing dependency injection, automatic OpenAPI documentation, and native WebSocket support [10].
- **PostgreSQL.** A relational database used for both the shop tables and the intelligence results, accessed through the SQLAlchemy ORM and the asyncpg driver.
- **React with Vite and Tailwind CSS.** Used to build the two single-page applications; Vite provides build tooling and Tailwind utility-first styling.
- **Scikit-learn, PyTorch, and lifetimes.** Provide the gradient-boosting classifier and preprocessing, the autoencoder, and the BG/NBD and Gamma-Gamma implementations [9], [11], [12].
- **Groq LLM API.** Hosts the large language model (qwen3-32b) used for schema-detection assistance, narrative insight generation, and natural-language querying.

2.2.3 Comparison

Table 2.1 positions the delivered system against representative existing tools. The distinguishing characteristic of Mindless System is that it unifies first-party behavioral capture, interpretable segmentation, predictive value modeling, and narrative insight within one self-hosted application.

Table 2.1: Comparison of Existing Systems with Mindless System

Capability	**Google Analytics / Mixpanel**	**Commercial CRM / CDP**	**Mindless System**
Behavioral event tracking	Yes (descriptive)	Partial	Yes, identity- and value-tagged
RFM segmentation	No (manual)	Partial	Yes, 11-segment rule model
CLV prediction	No	Sometimes	Yes (BG/NBD + Gamma-Gamma)
Anomaly detection	No	Rare	Yes (autoencoder / statistical)
Real-time monitoring	Limited	Limited	Yes (WebSocket stream)
Narrative LLM insights	No	Emerging	Yes (Groq LLM)
Coupled to the store	No	No	Yes (single application)

2.2.4 Research Findings

The review taught us two things that guided how we built the system. First, if people are actually going to use the tool, they need to understand it. Shop staff have to make real decisions based on customer segments, so we chose a simple rule-based RFM method (where you can see exactly why a customer landed in a group) instead of a "black box" clustering method that just spits out groups with no explanation.

Second, the value-prediction model (CLV) works well even when there isn't much purchase data, but it needs a certain number of repeat buyers before it gives trustworthy results , that's why we set minimum data limits before running it. And for spotting unusual customers, we added a simple backup method so the system keeps working even when the trained AI model isn't available.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Requirement Analysis

Requirements were derived from the project objectives and refined against the behavior of the working system. They divide into functional requirements, which describe what the system does, and non-functional requirements, which describe the qualities it must exhibit.

3.1.1 Functional Requirements

Table 3.1: Functional Requirements

ID	Requirement	Actor
FR-1	Register and authenticate customers; authenticate administrators through a role-restricted endpoint.	Customer, Admin
FR-2	Browse products by category and sub-category, search products, and view product detail pages.	Customer
FR-3	Add and remove items from a cart and a wishlist, and place an order via Cash on Delivery.	Customer
FR-4	Record each behavioral interaction as an analytics event tagged with user identity and monetary value.	System
FR-5	Create, update, and remove products with image upload; decrement stock on order.	Admin
FR-6	List orders and update order status, recording each change in an audit log.	Admin
FR-7	Stream behavioral events to the admin panel in real time over a WebSocket.	Admin, System
FR-8	Run the intelligence pipeline on an uploaded CSV or on the platform’s live order data.	Admin
FR-9	Compute RFM scores and segments, CLV, HVR potential, and anomaly classification per customer.	System
FR-10	Generate narrative insights and answer natural-language questions about an analyzed dataset.	Admin, System
FR-11	Report storefront engagement over a selectable time window: the most-visited pages and the most-wishlisted products.	Admin

3.1.2 Non-Functional Requirements

Table 3.2: Non-Functional Requirements

ID	Category	Requirement
NFR-1	Security	Passwords stored as bcrypt hashes; access controlled by signed JWTs and a role check; admin-only endpoints protected by a dependency.
NFR-2	Performance	Asynchronous request handling and a connection pool; indexed queries on events and orders for responsive analytics.
NFR-3	Reliability	The pipeline degrades gracefully: statistical anomaly fallback and null-valued predictions when data or models are insufficient.
NFR-4	Usability	Two focused single-page applications with clear navigation and immediate visual feedback.
NFR-5	Maintainability	Modular pipeline with one responsibility per module; clear separation of shop and intelligence concerns.
NFR-6	Scalability	Stateless API and pooled database access allow horizontal scaling of the web tier.

3.2 Feasibility Analysis

3.2.1 Technical Feasibility

The system is technically feasible: it is built entirely on mature, open-source technologies (Python, FastAPI, PostgreSQL, React, scikit-learn, and PyTorch) that are well documented and widely supported. The analytical methods (RFM, BG/NBD, autoencoders) are established and reliable, and development and testing were carried out on standard hardware.

3.2.2 Operational Feasibility

The system is operationally feasible. An operator can trigger a full analysis of live data with a single action and read the results as KPIs, segment distributions, and narrated insights, while behavioral capture is automatic and requires no manual instrumentation.

3.2.3 Economic Feasibility

The system is economically feasible. It relies on open-source frameworks with no licensing cost; the only external paid dependency is the optional LLM API, and the system remains functional without it by using rule-based insights.

3.2.4 Schedule Feasibility

The project was planned as a sequence of Agile iterations over the academic timeline, summarized in Table 3.3. Each phase produced a working, testable artifact, which kept the schedule realistic for a two-member team.

Table 3.3: Development Schedule

Phase	**Activities**	**Duration (approx.)**
Requirement analysis	Objective refinement, requirement gathering, technology selection	Weeks 1 to 2
System design	Architecture, database, and interface design; UML modeling	Weeks 3 to 4
Implementation, shop	Auth, catalogue, cart, wishlist, orders, behavioral tracking	Weeks 5 to 8
Implementation, intelligence	RFM, CLV, HVR, anomaly detection, insights, live tracking	Weeks 9 to 12
Testing	Unit, integration, and functional testing; bug fixing	Weeks 13 to 14
Documentation	Report writing and finalization	Weeks 15 to 16

3.3 Analysis

The system was analyzed using an object-oriented approach. Its use cases capture the functional interactions of the actors, while the dynamic and process models (the sequence and activity diagrams) describe the behavior of the key scenarios. The conceptual data model is expressed as an entity–relationship diagram.

3.3.1 Use Case Diagram

The system has three actors: the Customer, who interacts with the shop; the Administrator, who manages the catalogue and runs the intelligence engine; and the automated System, which tracks behavior and executes the machine-learning stages. Figure 3.1 presents the use case diagram, and Table 3.4 describes the key use cases.

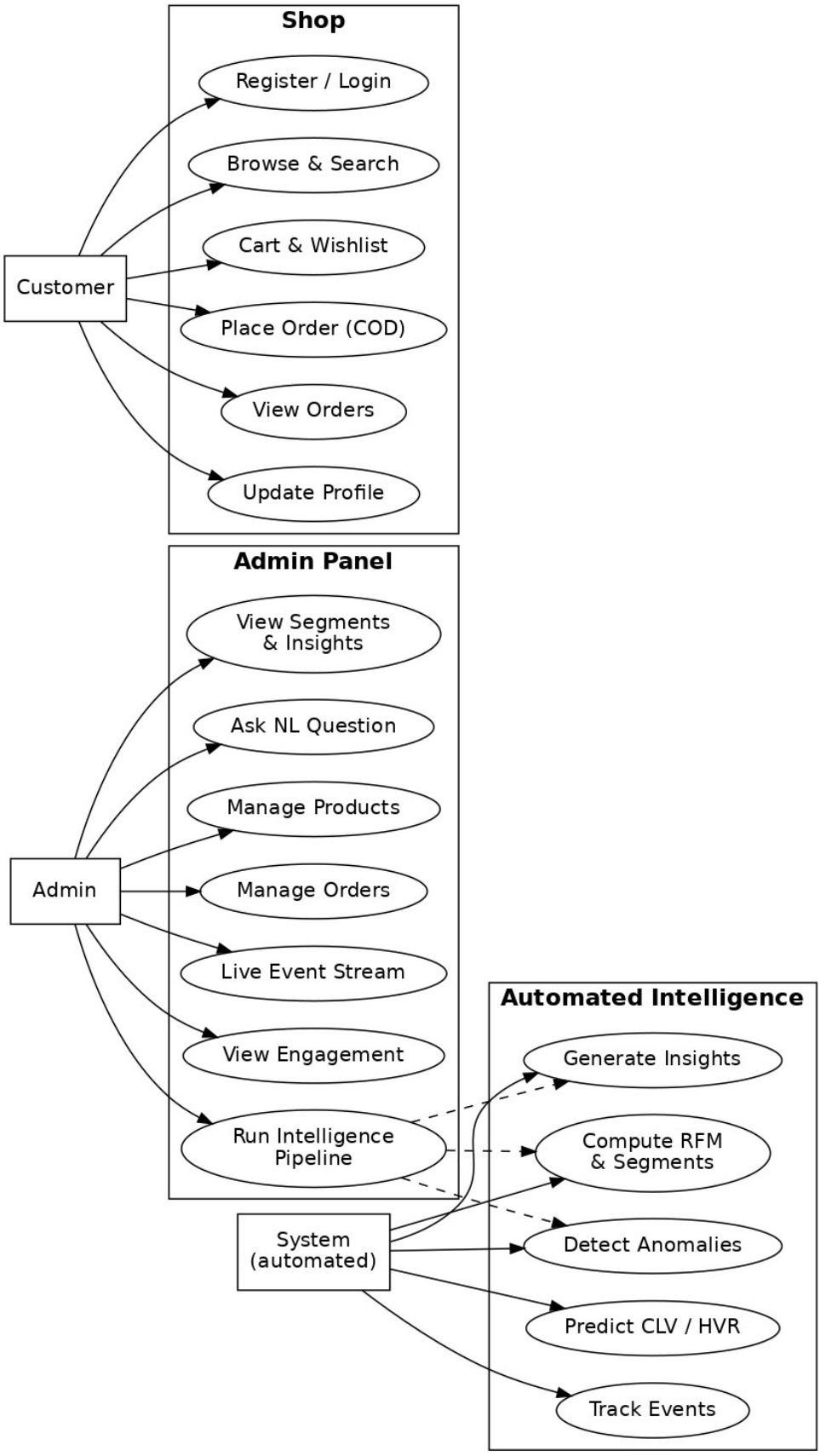


Figure 3.1: Use Case Diagram

Table 3.4: Key Use Case Descriptions

Use Case	Actor	Description
Place Order	Customer	The customer confirms the cart, provides delivery information, and places a Cash-on-Delivery order; the order is persisted, stock is decremented, and a purchase event is recorded.
Run Intelligence Pipeline	Admin	The admin triggers analysis on live order data; the backend exports line items to a CSV, creates a job, and executes the pipeline as a background task.
View Live Event Stream	Admin	The admin opens Live Tracking; the panel opens a WebSocket, is authorized as admin, and renders behavioral events as they arrive.
Ask NL Question	Admin	The admin submits a natural-language question about an analyzed job; the backend grounds an LLM prompt in the computed aggregates and returns an answer.
View Engagement	Admin	The admin opens the Engagement view and selects a time window; the panel reports the most-visited pages and the most-wishlisted products, aggregated from the stored analytics events.

3.3.2 Activity Diagram

Figure 3.2 depicts the control flow of the intelligence pipeline when an administrator analyzes the platform’s live users, including the authorization check, the data-volume guards on HVR and CLV, and the graceful-failure path when schema detection fails.

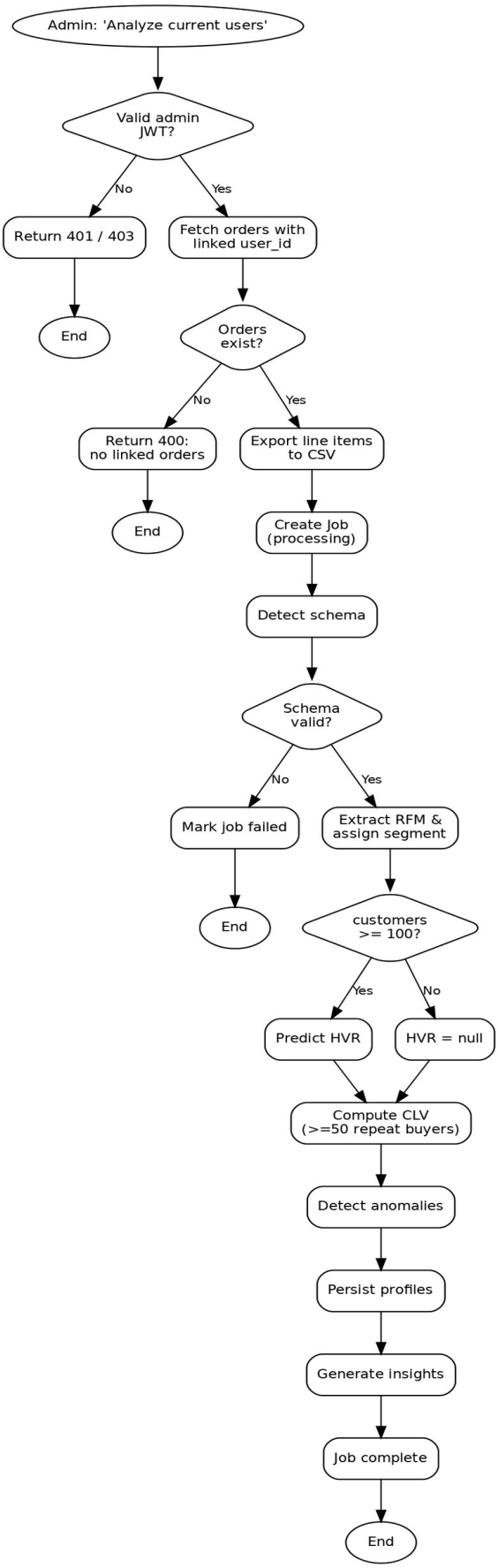


Figure 3.2: Activity Diagram of the Intelligence Pipeline

3.3.3 Sequence Diagram

Figure 3.3 shows the real-time event-tracking interaction. An administrator opens the live dashboard and the panel establishes an authorized WebSocket; when a customer interacts with the shop, the resulting event is persisted, published to the in-memory hub, and pushed to the subscribed dashboard.

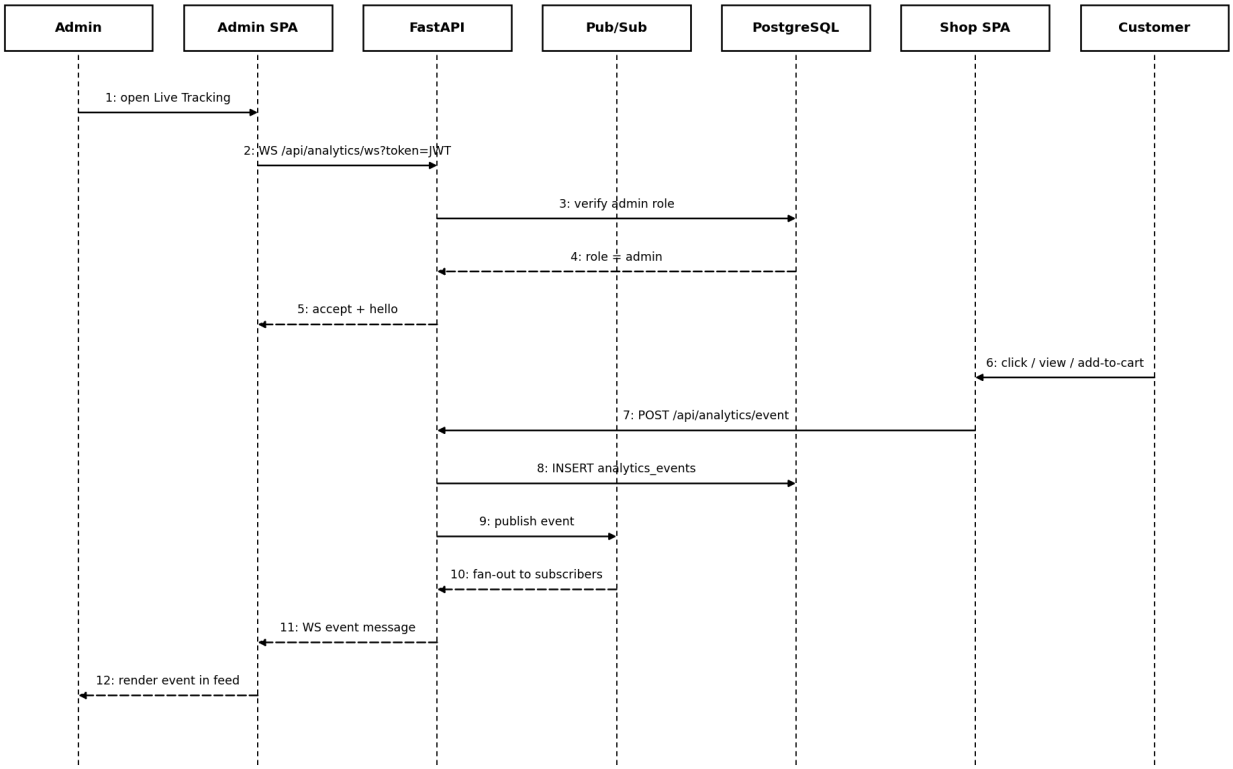


Figure 3.3: Sequence Diagram for Real-Time Event Tracking

3.3.4 Entity-Relationship Diagram

Figure 3.4 presents the conceptual data model. The shop entities (users, orders, products, analytics_events, audit_log) are managed through the asyncpg driver, while the intelligence entities (jobs, customer_profiles, insights) are managed through the SQLAlchemy ORM and keyed by a UUID job identifier.

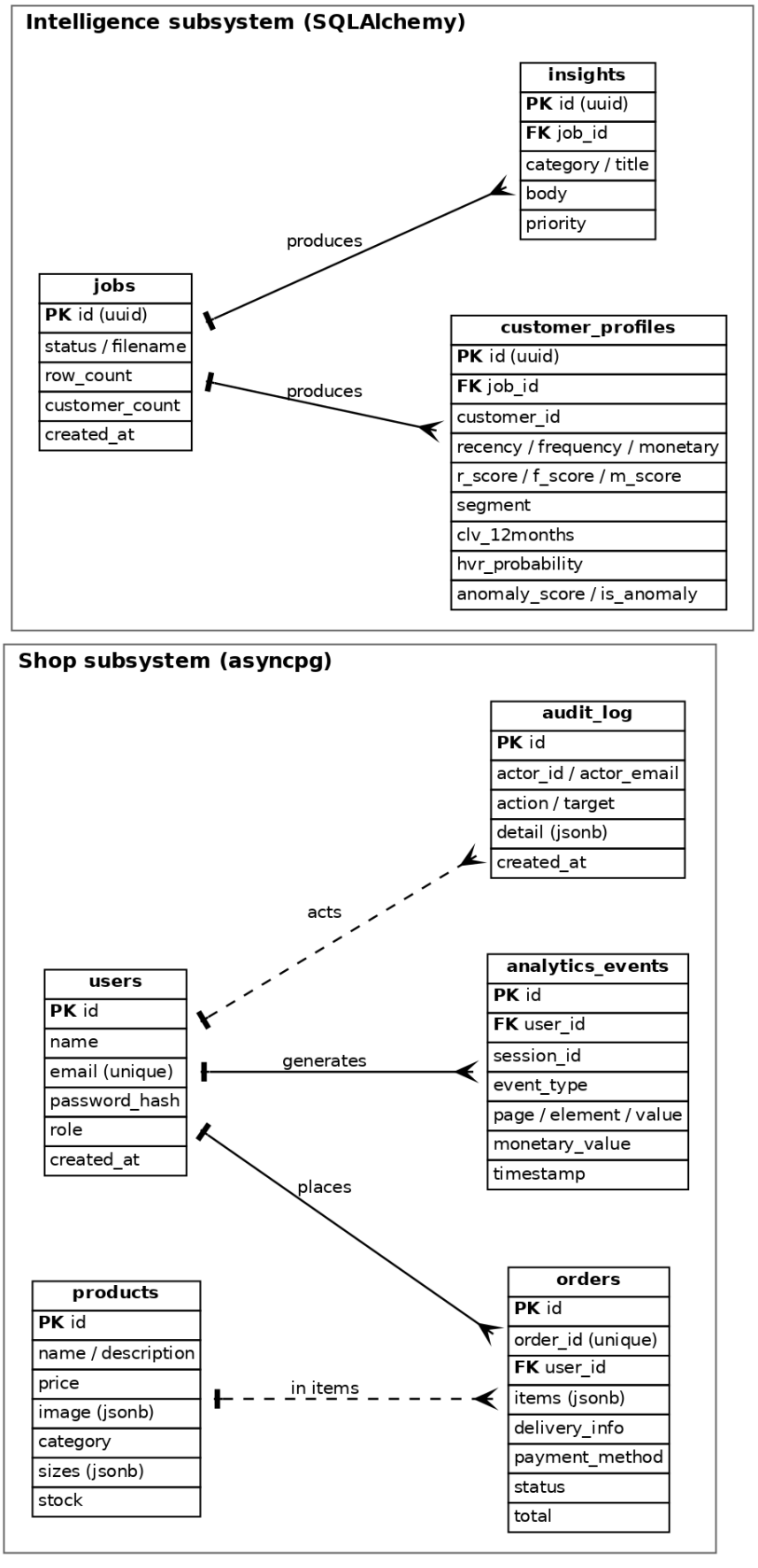


Figure 3.4: Entity-Relationship Diagram

3.4 User Roles

Table 3.5: User Roles and Privileges

Role	Privileges
Customer (user)	Register and log in; browse, search, cart, and wishlist; place and view own orders; update own profile. Cannot access admin or intelligence endpoints.
Administrator (admin)	All authenticated abilities plus product CRUD, order management, live tracking, and full access to the intelligence pipeline, results, insights, and natural-language query. Enforced by a database role check on every admin endpoint.

CHAPTER 4: SYSTEM DESIGN

4.1 Design

Because the system was analyzed with an object-oriented approach, the design refines the object-oriented models produced during analysis and adds the structural views required to implement the system. This section presents the system architecture, the refined class model, the state and component views, the deployment topology, and the database design derived from the conceptual model. The interaction and control-flow behavior established in analysis is preserved: the refined sequence and activity behavior of the system is as presented in Figures 3.3 and 3.2 respectively.

4.1.1 System Architecture

The system follows a layered client-server architecture. Two React single-page applications form the client layer. A FastAPI application forms the application layer and exposes REST endpoints and a WebSocket; it hosts the authentication dependencies, an in-memory publish/subscribe hub for live events, and the machine-learning pipeline orchestrator. A single PostgreSQL database forms the data layer, accessed through two drivers, SQLAlchemy for the intelligence tables and asyncpg for the shop tables. An external Groq LLM service is consulted for schema-detection assistance and insight generation. Figure 4.1 shows the overall architecture.

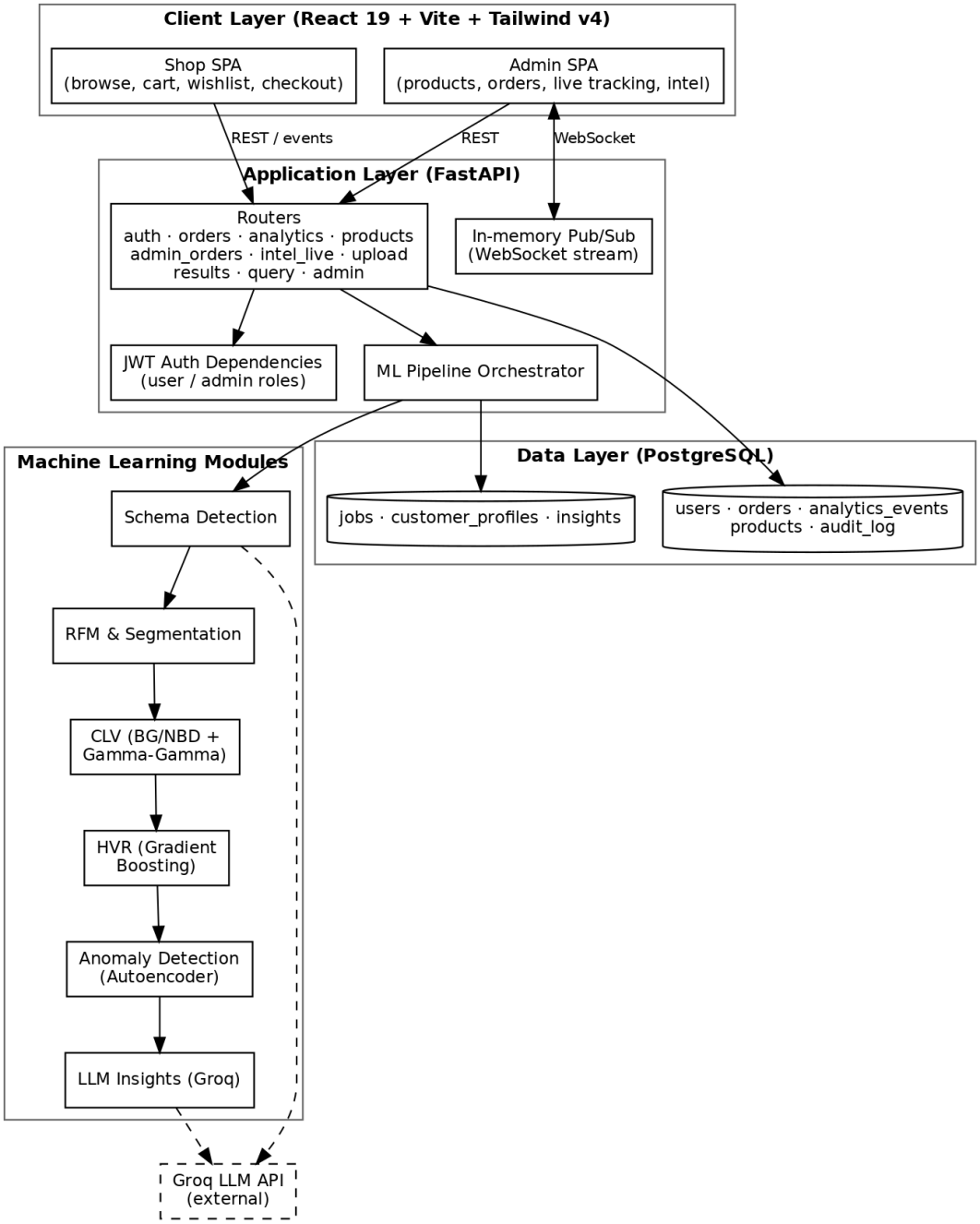


Figure 4.1: System Architecture of Mindless System

4.1.2 Refinement of Class and Object Diagrams

The design refines the object model into cohesive classes, one responsibility per class. The pipeline orchestrator depends on the analytical stage classes (schema detection, RFM extraction and segmentation, CLV, HVR prediction, anomaly detection (which owns the autoencoder), and insight generation) while the supporting classes provide database sessions and the connection pool, the in-memory live-tracking hub, the JWT dependencies,

and the audit helper. Figure 4.2 shows the refined class diagram of the machine-learning pipeline.

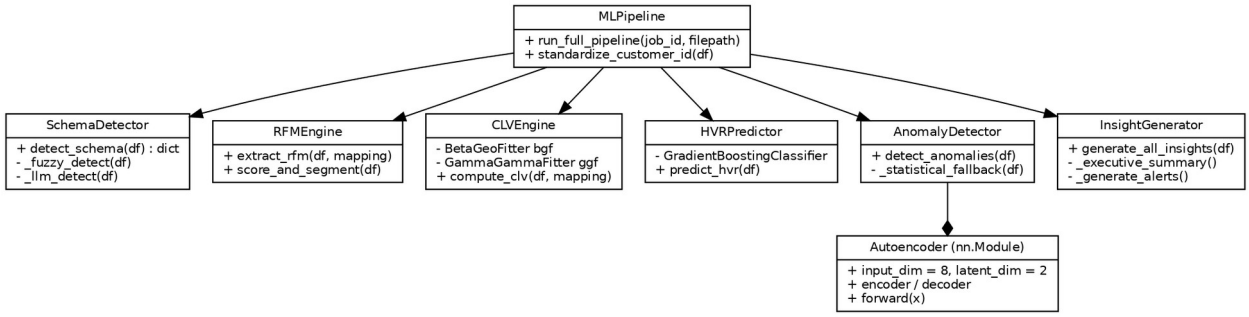


Figure 4.2: Class Diagram of the ML Pipeline Modules

4.1.3 State Diagram

An order is the central stateful entity in the system. Figure 4.3 models its lifecycle: a customer places a Cash-on-Delivery order, after which an administrator advances the order through Packing, Shipped, Out for Delivery, and finally Delivered. Each transition is an administrator action recorded in the audit log.

Figure 4.3: State Diagram of the Order Lifecycle

4.1.4 Component Diagram

Figure 4.4 shows the components of the system and their interfaces. The two single-page applications communicate with the FastAPI application, within which the authentication, shop-API, intel-API, live-tracking, and ML-pipeline components collaborate. All server components persist to a single PostgreSQL database, and the pipeline calls the external Groq LLM service.

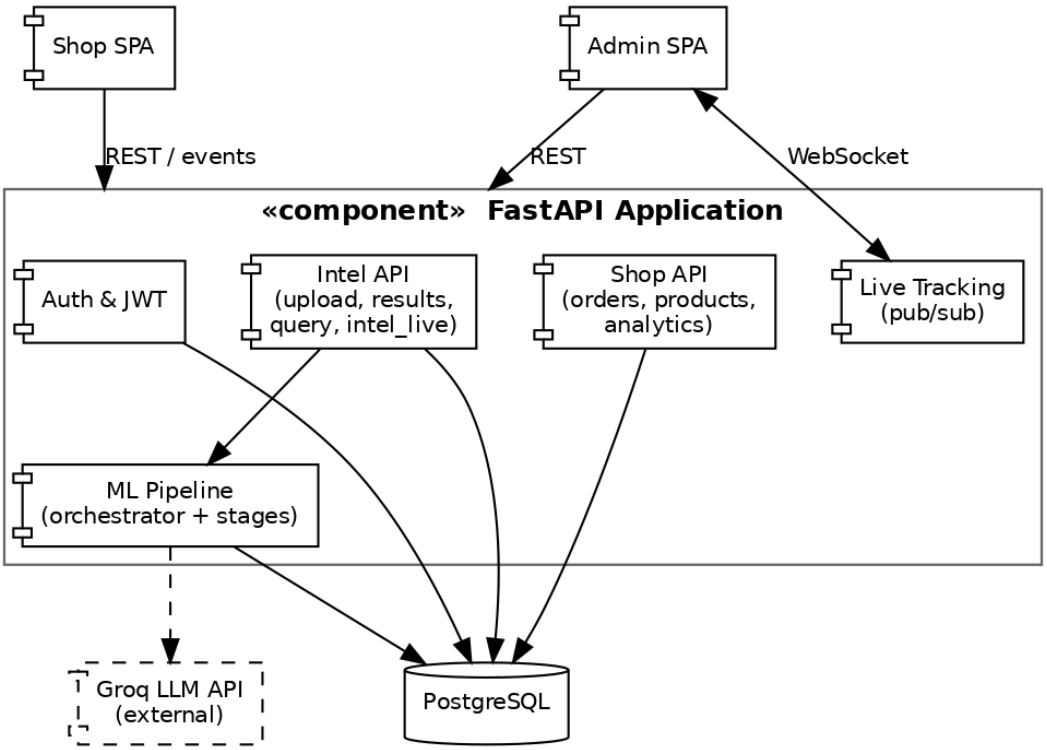


Figure 4.4: Component Diagram

4.1.5 Database Design

The conceptual entity–relationship model of Figure 3.4 is transformed into relational tables within a single PostgreSQL instance. The shop tables (created and migrated idempotently through the asyncpg driver) comprise users, orders, products, analytics_events, and audit_log, with indexes on event type, timestamp, session, and user, and on order user, status, and creation time. The intelligence tables (created through the SQLAlchemy ORM) comprise jobs, customer_profiles, and insights. Table 4.1 summarizes the principal tables.

Table 4.1: Principal Database Tables

Table	Driver	Purpose / Key Columns
users	asyncpg	Registered accounts; id, name, email (unique), password_hash, role (user/admin).
orders	asyncpg	Placed orders; order_id (unique), user_id (FK), items (JSONB), total, status, payment_method.
products	asyncpg	Catalogue; name, price, image (JSONB), category, sizes (JSONB), stock, bestseller.
analytics_events	asyncpg	Behavioral events; session_id, user_id, event_type, page, element, value, monetary_value, timestamp.
audit_log	asyncpg	Administrative actions; actor_id, actor_email,

Table	Driver	Purpose / Key Columns
		action, target, detail (JSONB).
jobs	SQLAlchemy	Pipeline runs; id (UUID), status, filename, row_count, customer_count, timestamps.
customer_profiles	SQLAlchemy	Per-customer results; RFM values and scores, segment, CLV, HVR, anomaly fields.
insights	SQLAlchemy	Generated insight cards; category, title, body, priority.

4.2 Algorithm Details

4.2.1 Customer Lifetime Value

Lifetime value is estimated with the BG/NBD model for the number of future transactions and the Gamma-Gamma model for average transaction value, both from the lifetimes library, using only repeat purchasers to fit the models.

The BG/NBD (Beta-Geometric / Negative Binomial Distribution) model is a probabilistic “buy-till-you-die” model of repeat-purchase behaviour. It represents each customer with two hidden processes. While a customer is active, orders arrive as a Poisson process whose rate varies across the population (a Gamma prior, which yields the negative-binomial transaction count); and after any purchase the customer may become permanently inactive with a per-customer dropout probability (a Beta prior, which yields the geometric dropout). Fitted by maximum-likelihood on each customer’s frequency (number of repeat purchases), recency (time between the first and last purchase) and T (observed age), the model returns both the expected number of future transactions and a probability that the customer is still active. A long gap since the last purchase lowers this probability, which is how the model infers silent churn without an explicit cancellation event.

The Gamma-Gamma model estimates the monetary value of a customer’s transactions. It assumes each customer has an unobserved average spend drawn from a Gamma distribution across the population, and that individual order values scatter around that average through a second Gamma process. Estimates for customers with few orders are shrunk towards the population mean, which prevents an unreliable average being drawn from a single large purchase. The model assumes that purchase frequency and monetary value are independent — an assumption the lifetimes library checks before fitting.

Multiplying the Gamma-Gamma expected order value by the BG/NBD expected transaction count, discounted over the horizon, produces the twelve-month customer lifetime value.

The lifetime-value computation, from clv.py, is shown below.

```
summary = summary_data_from_transaction_data(
    df, "CustomerID", "Date", monetary_value_col="LineTotal",
    observation_period_end=obs_date, freq="D")
repeat = summary[summary["frequency"] > 0]
if len(repeat) < 50:
    return null_clv          # too few repeat buyers to fit reliably

bgf = BetaGeoFitter(penalizer_coef=0.01)
bgf.fit(repeat["frequency"], repeat["recency"], repeat["T"])
ggf = GammaGammaFitter(penalizer_coef=0.01)
ggf.fit(repeat["frequency"], repeat["monetary_value"])

repeat["prob_alive"]   = bgf.conditional_probability_alive(
    repeat["frequency"], repeat["recency"], repeat["T"])
repeat["clv_12months"] = ggf.customer_lifetime_value(
    bgf, repeat["frequency"], repeat["recency"], repeat["T"],
    repeat["monetary_value"], time=12, freq="D", discount_rate=0.01)
```

4.2.2 High-Value-Repeater Prediction

The HVR stage engineers derived features (spend per day, orders per day, average inter-purchase gap, spend diversity, and basket value) and applies a pre-trained gradient-boosting classifier to estimate the probability that a customer becomes a high-value repeater, mapping the probability to Low, Medium, or High potential. The stage runs only when at least one hundred customers are present and the trained model artifacts exist on disk; the trained artifacts are bundled with the project.

The classifier is a gradient-boosting ensemble of decision trees. Boosting builds the trees sequentially: each shallow tree is fitted to the residual errors of the current ensemble and its scaled prediction is added to the running total, so successive trees progressively correct the mistakes of their predecessors. The trained model uses two hundred trees of depth three, a learning rate of 0.05 and 0.8 sub-sampling, and the minority high-value class is up-weighted during training to offset class imbalance. The shallow trees, sub-sampling and small learning rate together act as regularisation that limits over-fitting. The ensemble outputs a probability that a customer becomes a high-value repeater, which is then divided into Low, Medium, and High potential bands.

The prediction step, from prediction.py, is shown below.

```
df["monetary_per_day"] = df["Monetary"] / (df["TenureDays"] + 1)
df["basket_value"]     = df["Monetary"] / (df["TotalItems"] + 1)
# ... spend_diversity, orders_per_day, avg_gap similarly ...
X_scaled = scaler.transform(df[feature_cols])
df["hvr_probability"] = model.predict_proba(X_scaled)[:, 1]
df["hvr_potential"]   = pd.cut(df["hvr_probability"], [0, 0.33, 0.66, 1.0],
                        labels=["Low Potential", "Medium Potential", "High Potential"])
```

4.2.3 Anomaly Detection

Anomaly detection uses a PyTorch autoencoder that compresses eight behavioral features to a two-dimensional latent space and reconstructs them; the mean squared reconstruction error is the anomaly score, with the 95th and 99th percentiles defining Suspicious and High-Risk thresholds. A rule then classifies each flagged customer into an interpretable type. The trained autoencoder weights are bundled with the project and load by default; when the model directory is unavailable, the system falls back to a Z-score statistical method. The autoencoder is defined as follows.

An autoencoder is an unsupervised neural network trained to reproduce its own input through a narrow bottleneck. The encoder compresses the eight standardised behavioural features to a two-dimensional latent representation, and the decoder reconstructs the original eight features from it; forcing the data through only two dimensions makes the network learn the dominant, “normal” structure of the customer base. Because it is trained predominantly on typical customers, ordinary profiles are reconstructed accurately while unusual ones incur a large mean-squared reconstruction error, which is used directly as the anomaly score. No labelled anomalies are required, which is why an unsupervised method is appropriate for this task.

```
class Autoencoder(nn.Module):
    def __init__(self, input_dim=8, latent_dim=2):
        super().__init__()
        self.encoder = nn.Sequential(nn.Linear(input_dim,16), nn.ReLU(),
                                     nn.Linear(16,8), nn.ReLU(), nn.Linear(8,latent_dim))
        self.decoder = nn.Sequential(nn.Linear(latent_dim,8), nn.ReLU(),
                                     nn.Linear(8,16), nn.ReLU(), nn.Linear(16,input_dim))
    def forward(self, x):
        return self.decoder(self.encoder(x))
```

The detection logic that applies the autoencoder, from anomaly.py, is shown below.

```
X = RobustScaler().fit_transform(df[FEATURE_COLS].fillna(0)).astype("float32")
model = Autoencoder(input_dim=8, latent_dim=2)
model.load_state_dict(torch.load(model_path, map_location=device))
model.eval()
with torch.no_grad():
   recon = model(torch.tensor(X)).cpu().numpy()
errors = np.mean((X - recon) ** 2, axis=1)          # reconstruction error

t95, t99 = np.percentile(errors, 95), np.percentile(errors, 99)
df["anomaly_score"]    = errors
df["is_anomaly"]       = (errors > t95).astype(int)
df["anomaly_severity"] = np.where(errors > t99, "High Risk",
                         np.where(errors > t95, "Suspicious", "Normal"))
```

4.2.4 Schema Detection and Insight Generation

A schema-detection stage maps arbitrary columns to the canonical customer-id, date, amount, quantity, and invoice fields using fuzzy string matching with an optional LLM pass. After the numerical stages complete, an insight generator produces an executive summary, per-segment recommendations, and anomaly commentary through the LLM, alongside deterministic rule-based priority alerts that are always available.

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Implementation

5.1.1 Tools Used

The programming languages used are Python (backend and machine learning), JavaScript with JSX (the two React front-ends), and SQL (database access), and the database platform is PostgreSQL. Supporting CASE and development tools included Visual Studio Code as the editor, Git for version control, Graphviz for the UML and system diagrams, and the interactive Swagger UI (OpenAPI) for exercising and documenting the API. Table 5.1 summarizes the full set of tools, frameworks, and libraries.

Table 5.1: Tools and Technologies Used

Category	**Tools / Technologies**
Programming languages	Python 3.11, JavaScript (JSX), SQL.
Backend framework	FastAPI, Uvicorn (ASGI server), Pydantic v2, SQLAlchemy 2, asyncpg, PyJWT, bcrypt, python-decouple / python-dotenv.
Machine learning	scikit-learn, PyTorch, lifetimes (BG/NBD, Gamma-Gamma), pandas, NumPy, SciPy, rapidfuzz.
Large language model	Groq API (qwen/qwen3-32b) for schema-detection assistance, insights, and NL query.
Frontend	React 19, Vite, Tailwind CSS v4, React Router 7, Recharts (shop), Axios (admin), React-Toastify.
Database platform	PostgreSQL 14+ (single database, dual driver: SQLAlchemy and asyncpg).
CASE / development tools	Visual Studio Code, Git,Swagger UI / OpenAPI (API documentation and testing).

5.1.2 Implementation Details of Modules

The backend is a single FastAPI application defined in app/main.py. A lifespan handler runs at startup: it creates the SQLAlchemy intelligence tables and initializes the asyncpg pool, which idempotently creates and migrates the shop tables using CREATE TABLE IF NOT EXISTS and ALTER TABLE ADD COLUMN IF NOT EXISTS statements. The

HTTP and WebSocket surface is organized into router modules (auth, orders, analytics, products, admin_orders, intel_live, upload, results, query, and admin), each mounted under the common /api prefix.

The machine-learning pipeline is implemented as one module per stage. The orchestrator function run_full_pipeline(job_id, filepath) in the services layer reads the input data and then calls detect_schema, extract_rfm and score_and_segment, predict_hvr, compute_clv, detect_anomalies, and generate_all_insights in sequence; it persists a CustomerProfile object for every customer with bulk_save_objects and writes the generated Insight cards. The anomaly stage owns the Autoencoder class, an nn.Module with an eight-to-two encoder and a symmetric decoder. The algorithms executed by these functions are described in Section 4.2.

Authentication and authorization are implemented in auth_deps.py and routers/auth.py. On login the server verifies the submitted password against a bcrypt hash and issues a signed JWT; the get_current_user_id dependency validates the token, while get_current_admin additionally verifies the user role against the database and rejects non-administrators with HTTP 403, as shown below.

```
def _create_token(user_id, email):
    payload = {"sub": str(user_id), "email": email,
               "exp": datetime.now(timezone.utc) + TOKEN_EXPIRY}
    return jwt.encode(payload, JWT_SECRET, algorithm=JWT_ALGO)

async def get_current_admin(user_id = Depends(get_current_user_id), conn = ...):
    role = await conn.fetchval("SELECT role FROM users WHERE id = $1", user_id)
    if role != "admin":
        raise HTTPException(status_code=403, detail="Admin access required")
    return user_id
```

Real-time delivery is implemented by an in-memory publish/subscribe hub (live_tracking.py) that maintains a set of asyncio queues; the analytics-event endpoint publishes each event to the hub, and the WebSocket endpoint streams events to authorized admin subscribers. Table 5.2 lists the principal endpoints exposed by these modules; interactive OpenAPI documentation is available at /docs.

Table 5.2: Principal REST and WebSocket API Endpoints

Method & Path	Access	Purpose
POST /api/auth/signup, /login	Public	Register or authenticate a customer and return a JWT.
POST /api/auth/admin/login	Public	Authenticate an administrator (role-checked).
POST /api/analytics/event	Public	Ingest a behavioral analytics event.
POST /api/orders/	Public/ User	Place an order; links user id from JWT when present.
GET /api/product/list, /{id}	Public	List products or fetch a product.
GET /api/auth/me, PATCH /api/auth/profile	User	Read or update the current profile.
POST /api/upload	Admin	Upload a CSV and launch the ML pipeline.
POST /api/intel/run-on-current-users	Admin	Run the pipeline on live order data.
GET /api/results/{job}/overview, /customers, /insights	Admin	Fetch pipeline results.
POST /api/results/{job}/query	Admin	Answer a natural-language question.
GET /api/analytics/summary, /segments, /live	Admin	KPIs, live segmentation, recent-activity snapshot.
POST /api/admin/train	Admin	Retrain and persist the HVR gradient-boosting model.
WS /api/analytics/ws	Admin	Subscribe to the real-time event stream.
GET /api/analytics/engagement	Admin	Most-visited pages and most-wishlisted products over a time window.

On the client, the shop application implements passive behavioral tracking through the useAnalytics hook, which attaches listeners for clicks, hovers, key presses, and mouse movement and emits identity- and value-tagged events. The admin application implements the product and order management pages, a live-tracking page backed by a WebSocket, an engagement page that reports the most-visited pages and most-wishlisted products, and an intelligence workspace that renders KPIs, segment distributions, top customers, insights, and a natural-language query box. Input validation is performed by Pydantic schemas at the API boundary; the pipeline is wrapped in exception handling that marks a job failed

and records the error rather than crashing; administrative state changes are recorded in an append-only audit log; and secrets are loaded from environment variables and excluded from version control.

A dedicated read-only endpoint backs the engagement view. It aggregates the stored analytics events without touching the machine-learning pipeline: page-view counts are grouped by page, while wishlist activity is reduced to a net count (additions minus removals) per product and joined to the product catalogue to recover names, prices, and thumbnails. Because it reuses the existing analytics_events and products tables, the feature adds reporting value with no change to the database schema or the intelligence models.

The implemented user interface is realized as two single-page applications. The following figures indicate where screenshots captured from the running application should be inserted; the captions describe the implemented screens.

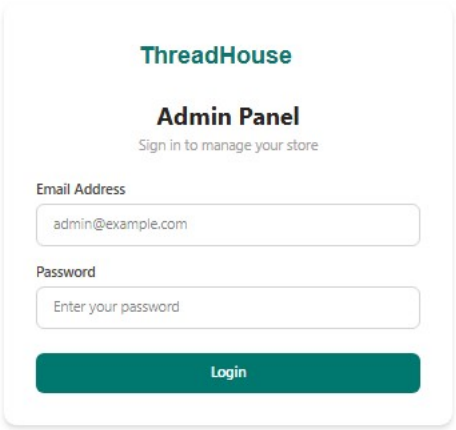


Figure 5.1: Admin Login Page

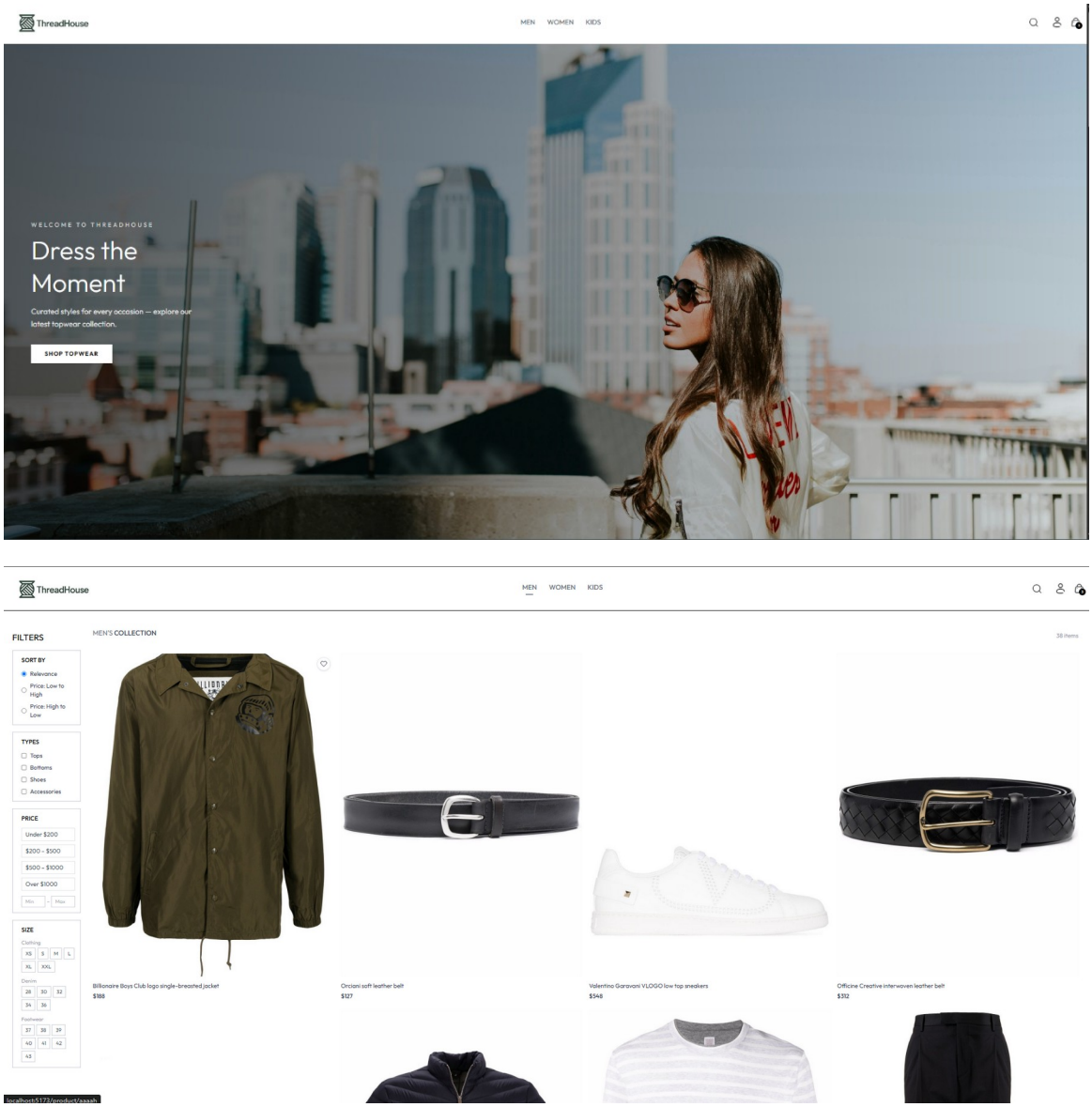


Figure 5.2: Shop Home / Product Listing

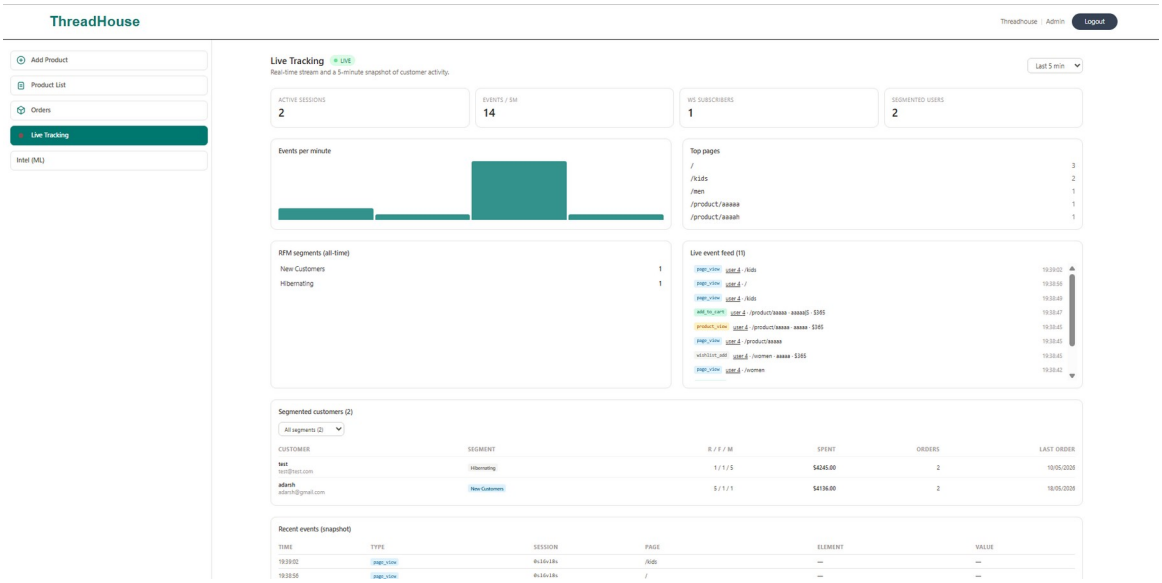


Figure 5.3: Live Tracking Dashboard

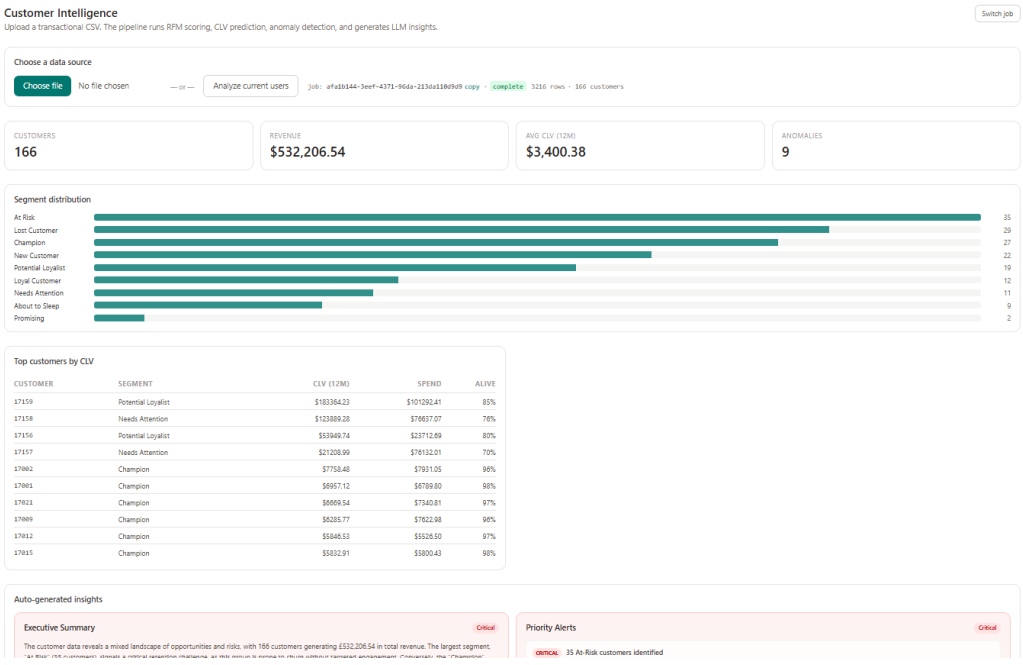


Figure 5.4: Intelligence Dashboard (Segments and KPIs)

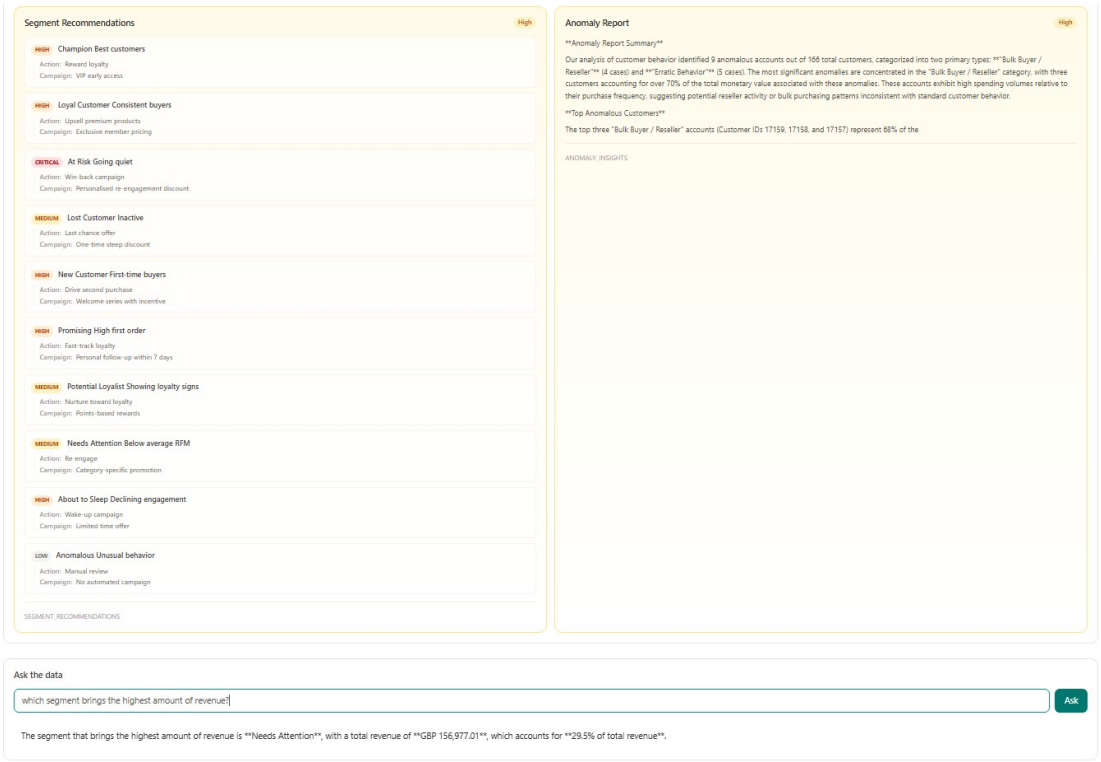


Figure 5.5: Insights and Natural-Language Query

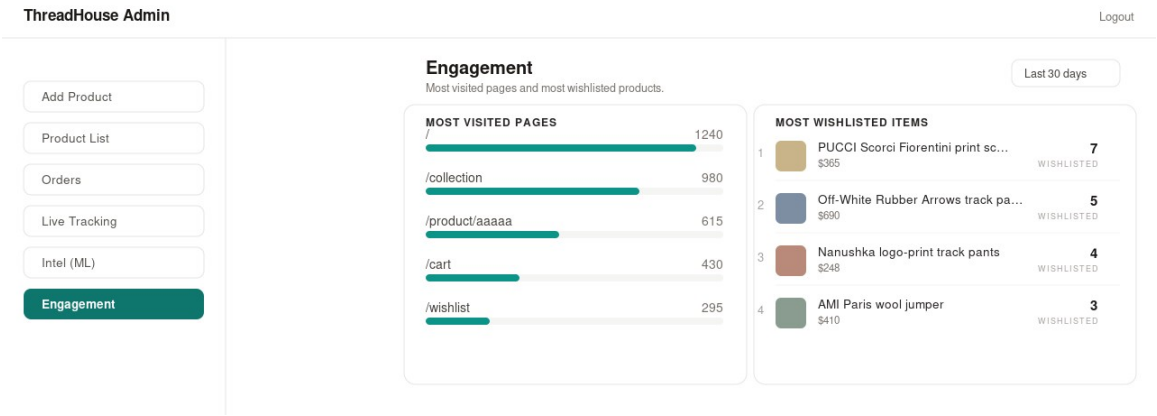


Figure 5.6: Engagement View (Most Visited Pages and Most Wishlisted Products)

5.2 Testing

Testing was carried out at two levels. Unit testing verified individual analytical functions in isolation, with attention to the small-sample edge cases that the scoring and lifetime-value routines are designed to survive. System testing verified complete user journeys and the cooperation of modules across their boundaries, exercising the shop, the administrative

panel, and the intelligence engine end to end. Every test case is recorded with its input test data, the expected result, and the observed (actual) result.

5.2.1 Test Cases for Unit Testing

Table 5.3: Test Cases for Unit Testing

S. N	Test Case ID	Test Description	Input Test Data	Expected Result	Actual Result	Pass/ Fail
1	TC-U01	RFM quintile scoring on a normal sample	100 distinct Monetary values	R, F, M scores mapped to 1 to 5	Scores returned within 1 to 5	Pass
2	TC-U02	Quintile scoring on identical values	A feature whose values are all 100	Default score returned without error	Default score 3 assigned, no crash	Pass
3	TC-U03	Segment assignment for a top customer	R = 5, F = 5, M = 5	Segment = Champion	Champion	Pass
4	TC-U04	Suspicious customer separation	Monetary = 0	Segment = Anomalous, scores = 0	Anomalous, scores 0	Pass
5	TC-U05	CLV with too few repeat buyers	20 repeat purchasers (fewer than 50)	Null CLV values, no exception	Null CLV returned	Pass
6	TC-U06	Anomaly detection without the model file	MODEL_DIR pointing to a missing path	Statistical Z-score fallback runs	Fallback executed	Pass
7	TC-U07	HVR with too few customers	40 customers (fewer than 100)	HVR fields set to null	Null HVR fields	Pass

5.2.2 Test Cases for System Testing

System testing was organized by feature. The login flow, the shop and order-management journey, and the administrative and intelligence features were each exercised with valid, boundary, and negative inputs, as recorded in Tables 5.4 to 5.6.

a. Test Case for Login

Table 5.4: Test Case for Login

Test Case ID	Test Description	Input Test Data	Expected Result	Actual Result	Pass/Fail
TC-01	Open the admin panel and load the login page	http://localhost:5174	Login page displayed with email and password fields	Login page displayed as expected	Pass
TC-02	Enter valid admin credentials	Email: admin@threadhouse.com Password: Admin@123	Redirect to dashboard; JWT stored	Dashboard displayed, token stored	Pass
TC-03	Enter a valid email with an empty password	Email: admin@threadhouse.com Password: (empty)	Error: password is required	Validation error shown, login blocked	Pass
TC-04	Submit with empty email and password	Email: (empty) Password: (empty)	Error: email and password are required	Validation error displayed	Pass
TC-05	Enter invalid credentials	Email: false@mail.com Password: falsepass	Error: invalid credentials (HTTP 401)	401 Invalid credentials shown	Pass
TC-06	A non-admin user attempts the admin login	Email: adarsh@gmail.com (role user) Password: Adarsh123	Error: admin access required (HTTP 403)	403 Admin access required shown	Pass

b. Test Case for Shop and Order Management

Table 5.5: Test Case for Shop and Order Management

Test Case ID	Test Description	Input Test Data	Expected Result	Actual Result	Pass/Fail
TC-07	Browse and search products	Category: Men; Search: shirt	Matching products listed	Filtered products displayed	Pass
TC-08	Add an item to the cart and wishlist	Click add-to-cart on a product	Item added; analytics event recorded	Cart updated, event sent	Pass
TC-09	Place a Cash-on-Delivery order while logged in	items[], delivery_info, payment_method = COD	HTTP 201; order persisted (Order Placed) and linked to the user	Order created and linked	Pass
TC-10	Place an order with a duplicate order id	order_id of an existing order	HTTP 409 duplicate order_id	409 returned	Pass
TC-11	Update an order status as admin	orderId, status = Shipped	Status updated; audit_log row written	Status changed and audited	Pass

c. Test Case for the Admin and Intelligence Engine

Table 5.6: Test Case for the Admin and Intelligence Engine

Test Case ID	Test Description	Input Test Data	Expected Result	Actual Result	Pass/Fail
TC-	Open Live	Admin opens Live	Events	Live feed	Pass

Test Case ID	Test Description	Input Test Data	Expected Result	Actual Result	Pass/Fail
12	Tracking and observe events	Tracking; customer clicks in the shop	appear in real time over the WebSocket	updated instantly	
TC-13	Open the WebSocket with an expired token	Expired JWT as the ws token parameter	Connection rejected before accept	Connection rejected	Pass
TC-14	Run the pipeline with no linked orders	Click Analyze current users with no linked orders	HTTP 400 with an explanatory message	400 returned with a message	Pass
TC-15	Run the pipeline on live order data	Click Analyze current users with orders present	Job created; pipeline completes; results populate	Job completed; KPIs and segments shown	Pass
TC-16	View the analysis results	GET /api/results/{job}/overview	KPIs and segment distribution returned	Data displayed on the dashboard	Pass
TC-17	Ask a natural-language question	Which segments drive the most revenue?	Grounded answer citing the analyzed data	Answer returned from the dataset	Pass
TC-18	Access admin analytics with a customer token	Customer JWT on /api/analytics/segments	HTTP 403 admin access required	403 returned	Pass
TC-19	Open the Engagement	Admin opens Engagement; 30-day window	Most-visited pages and	Lists rendered	Pass

Test Case ID	Test Description	Input Test Data	Expected Result	Actual Result	Pass/Fail
	view and select a window		most-wishlisted products are listed	with counts and thumbnails	

5.3 Result Analysis

For each analyzed job the system produces a set of per-customer profiles and a collection of insight cards. A representative run yields a segment distribution counting customers across the eleven segments; a ranked list of top customers by monetary value with their RFM codes; lifetime-value and high-value-repeater estimates where the data-volume thresholds are met; anomaly flags with an interpretable type for each flagged customer; and a large-language-model executive summary accompanied by rule-based priority alerts. These outputs are surfaced on the intelligence dashboard (Figure 5.4) and the insights view (Figure 5.5).

The testing summarized in Section 5.2 confirmed correct behavior across both levels: every recorded unit and system test case passed. With the bundled model artifacts the PyTorch autoencoder and the gradient-boosting classifier loaded and executed on live order data of modest size; the pipeline correctly returned null-valued predictions where the data-volume thresholds were not met, and the statistical anomaly fallback was verified by running with the model directory unavailable, confirming the graceful-degradation requirement. The authorization boundary behaved correctly, rejecting customer tokens on administrative routes and rejecting invalid or expired tokens on the WebSocket.

Defects identified during testing were resolved. Notable examples include quintile scoring crashing on very small samples (fixed by a robust binning fallback), CLV bucketing raising on near-identical values (fixed by adaptive bucket counts), and repeated unauthorized requests looping on the client when a token expired (fixed by intercepting HTTP 401 responses to clear the token and redirect to login). Dependency and environment issues encountered during setup, including an invalid package pin and a missing configuration package, were also corrected so that a clean installation succeeds.

5.3.1 Model Evaluation (HVR Classifier)

Of the analytical models, the high-value-repeater (HVR) classifier is the only supervised component, so it is the only one for which classification accuracy can be measured. It was evaluated on a temporal holdout test set (the customers whose first purchase falls in the most recent portion of the training window), which prevents the model from being scored on customers it effectively learned from. The results, obtained through the training routine described in Chapter 4 and exposed by the POST /api/admin/train endpoint, are reported in Table 5.7, and the confusion matrix is shown in Figure 5.7.

Table 5.7: HVR Classifier Evaluation Metrics

Metric	**Value**
ROC-AUC	0.853
Accuracy	0.805
Precision	0.500
Recall	0.615
F1-score	0.552

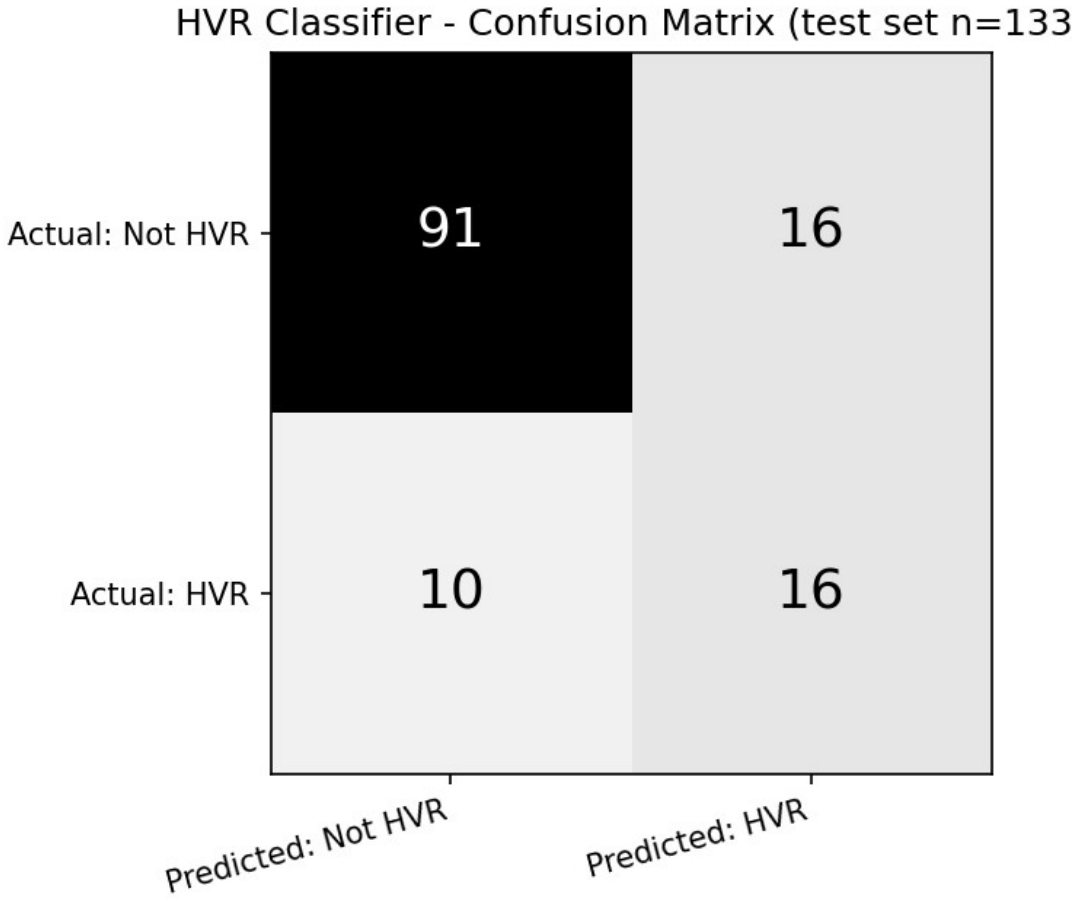


Figure 5.7: HVR Classifier Confusion Matrix (test set)

The classifier attains an ROC-AUC of 0.85, indicating a strong ability to rank future high-value repeaters ahead of the rest. From the confusion matrix, of the 26 actual high-value repeaters the model correctly identifies 16 (recall 0.62), while keeping false positives modest at 16 of 107 non-repeaters. Because high-value repeaters are a minority class, precision (0.50) is lower than overall accuracy (0.81); the training routine applies class weighting to partially offset this imbalance. The remaining models are not scored with classification metrics by their nature: anomaly detection is unsupervised, customer-lifetime-value estimation is probabilistic, and segmentation is rule-based and deterministic.

CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATIONS

6.1 Conclusion

The project delivered a complete, working system that meets its objectives. A functioning e-commerce platform captures genuine behavioral data; an administrative intelligence engine transforms that data into interpretable RFM segments, lifetime-value and high-value-repeater predictions, and anomaly classifications; and a large language model narrates the results and answers natural-language questions. A real-time WebSocket dashboard lets an operator watch behavior as it happens, and the whole pipeline can be launched against live data with a single action. Integrating both the data-producing and the data-analyzing halves in one application is the project's principal achievement, and it demonstrates that behavioral tracking, interpretable segmentation, probabilistic value modeling, anomaly detection, and language-model narration can be composed into a single coherent application.

The work also yielded practical lessons. Interpretability can outweigh sophistication (an explainable RFM rule model proved far more actionable for a business audience than opaque clustering. Real analytical pipelines must be defensive, because small samples, missing artifacts, and absent API keys are the norm rather than the exception, so every stage was designed to degrade gracefully. A clean separation of concerns, with one module per analytical stage and a clear split between shop and intelligence, kept the system easy to test and extend, and end-to-end integration surfaced issues that unit testing alone would not have revealed. The system's limitations) Cash-on-Delivery only, stateless tokens without revocation, data-volume thresholds on the predictive stages, and dependence on an external LLM for narrative features, reflect the scope of a coursework project rather than architectural obstacles.

6.2 Future Recommendations

- Integrate an online payment gateway alongside Cash on Delivery.
- Automate retraining and versioning of the trained autoencoder and gradient-boosting artifacts, which are currently bundled with the project, and add a scheduled retraining pipeline.

- Introduce token revocation and comprehensive rate limiting, and move secrets to a managed vault.
- Add automated regression tests and continuous integration.
- Extend the intelligence engine with churn-probability modeling and per-segment recommendation, building on the existing feature set.

REFERENCES

[1] Google, "Google Analytics," Google Marketing Platform. [Online]. Available: https://analytics.google.com

[2] Mixpanel, "Product Analytics Documentation," Mixpanel Inc. [Online]. Available: https://docs.mixpanel.com

[3] D. Ferreira, "Recommendation System Using Autoencoders," Applied Sciences, vol. 10, no. 16, 2020. [Online]. Available: https://www.mdpi.com/2076-3417/10/16/5510

[4] R. Wirth and J. Hipp, "CRISP-DM: Towards a Standard Process Model for Data Mining," in Proc. 4th Int. Conf. on Practical Applications of Knowledge Discovery and Data Mining, 2000.

[5] K. Schwaber and J. Sutherland, "The Scrum Guide: The Definitive Guide to Scrum," 2020. [Online]. Available: https://scrumguides.org

[6] P. Baldi, "Autoencoders, Unsupervised Learning, and Deep Architectures," in Proc. ICML Workshop on Unsupervised and Transfer Learning, 2012. [Online]. Available: https://proceedings.mlr.press/v27/baldi12a.html

[7] P. Fader, B. Hardie, and K. Lee, "RFM and CLV: Using Iso-Value Curves for Customer Base Analysis," Journal of Marketing Research, vol. 42, no. 4, pp. 415–430, 2005.

[8] P. Fader, B. Hardie, and K. Lee, "“Counting Your Customers” the Easy Way: An Alternative to the Pareto/NBD Model," Marketing Science, vol. 24, no. 2, pp. 275–284, 2005.

[9] C. Davidson-Pilon, "lifetimes: Measuring Customer Lifetime Value in Python." [Online]. Available: https://lifetimes.readthedocs.io

[10] S. Ramírez, "FastAPI Documentation." [Online]. Available: https://fastapi.tiangolo.com

[11] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[12] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Advances in Neural Information Processing Systems 32, 2019.

[13] The PostgreSQL Global Development Group, "PostgreSQL 14 Documentation." [Online]. Available: https://www.postgresql.org/docs

[14] Meta Open Source, "React Documentation." [Online]. Available: https://react.dev

[15] Encode, "Uvicorn: An ASGI Web Server for Python." [Online]. Available: https://www.uvicorn.org

[16] Tailwind Labs, "Tailwind CSS Documentation." [Online]. Available: https://tailwindcss.com/docs

[17] Evan You and the Vite Team, "Vite Documentation." [Online]. Available: https://vitejs.dev

[18] W. McKinney, "Data Structures for Statistical Computing in Python," in Proc. 9th Python in Science Conf. (SciPy), pp. 51–56, 2010.

[19] C. R. Harris et al., "Array Programming with NumPy," Nature, vol. 585, pp. 357–362, 2020.

[20] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, 2015.

[21] N. Provos and D. Mazieres, "A Future-Adaptable Password Scheme," in Proc. USENIX Annual Technical Conf., 1999.

APPENDICES

Appendix A: Source Code Snippets

Representative excerpts from the implementation are shown below. They are lightly abbreviated for space; the full source is available in the project repository.

F.1 Application startup and lifespan (app/main.py)

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    try:
        from app.db import models
        Base.metadata.create_all(bind=engine)
        print("SQLAlchemy tables ready: jobs, customer_profiles, insights.")
    except Exception as e:
        print(f"SQLAlchemy startup error: {e}")
    try:
        await init_asyncpg_pool()
    except Exception as e:
        print(f"asyncpg startup error: {e}")
        raise
    try:
        yield
    finally:
        await close_asyncpg_pool()

app = FastAPI(
    title="ThreadHouse + Customer Intelligence Engine",
    version="1.0.0", lifespan=lifespan, docs_url="/docs", redoc_url="/redoc")
app.add_middleware(
    CORSMiddleware,
    allow_origin_regex=r"http://(localhost|127.0.0.1):\d+",
    allow_credentials=False, allow_methods=["*"], allow_headers=["*"])
```

F.2 Pipeline orchestrator (app/services/ml_services.py)

```
def run_full_pipeline(job_id, filepath):
    db = SessionLocal()
    try:
        df_raw = pd.read_csv(filepath)
        job = db.query(Job).filter(Job.id == job_id).first()
        job.row_count = len(df_raw); db.commit()

        schema = detect_schema(df_raw)
        if not schema["success"]:
            raise ValueError("Schema detection failed: " + str(schema["missing"]))

        rfm_df    = standardize_customer_id(extract_rfm(df_raw, schema["mapping"]))
        scored_df = standardize_customer_id(score_and_segment(rfm_df))

        if len(scored_df) >= 100:
            scored_df = predict_hvr(scored_df)
        else:
            scored_df["hvr_probability"] = None
            scored_df["hvr_potential"]   = None

        clv_df    = standardize_customer_id(compute_clv(df_raw, schema["mapping"]))
        scored_df = scored_df.merge(clv_df, on="CustomerID", how="left")
```

```
        scored_df = detect_anomalies(scored_df)
        job.customer_count = len(scored_df); db.commit()

        profiles = [CustomerProfile(
            job_id=job_id, customer_id=str(row["CustomerID"]),
            recency=float(row["Recency"]), frequency=float(row["Frequency"]),
            monetary=float(row["Monetary"]), segment=str(row.get("Segment", "Unknown")),
            r_score=int(row.get("R_score", 0)), f_score=int(row.get("F_score", 0)),
            m_score=int(row.get("M_score", 0)),
            is_anomaly=bool(row.get("is_anomaly", False)),
            anomaly_type=str(row.get("anomaly_type", "Normal")),
        ) for _, row in scored_df.iterrows()]
        db.bulk_save_objects(profiles); db.commit()

        insights = generate_all_insights(scored_df)
        db.bulk_save_objects([Insight(job_id=job_id, category=c,
            title=v.get("title", c), body=v.get("body", str(v)),
            priority=v.get("priority", 3)) for c, v in insights.items()])
        db.commit()

        job.status = "complete"; job.completed_at = datetime.utcnow(); db.commit()
        save_plots(job_id, profiles)          # render + save chart PNGs
    except Exception as e:
        job.status = "failed"; job.error_message = str(e); db.commit()
    finally:
        db.close()
```

F.3 RFM feature extraction (app/pipeline/rfm_extraction.py)

```
def extract_rfm(df, mapping):
    df = df.rename(columns={
        mapping["customer_id"]["column"]: "CustomerID",
        mapping["date"]["column"]:        "Date",
        mapping["amount"]["column"]:      "Amount",
        mapping["quantity"]["column"]:    "Quantity",
        mapping["invoice_id"]["column"]:  "InvoiceID"})
    df["Date"]      = pd.to_datetime(df["Date"])
    df              = df[df["Quantity"] > 0]
    df              = df[df["Amount"] > 0]
    df["LineTotal"] = df["Quantity"] * df["Amount"]
    df              = df.dropna(subset=["CustomerID"])

    reference_date = df["Date"].max() + pd.Timedelta(days=1)
    rfm = df.groupby("CustomerID").agg(
        Recency          = ("Date",      lambda x: (reference_date - x.max()).days),
        Frequency        = ("InvoiceID", "nunique"),
        Monetary         = ("LineTotal", "sum"),
        AvgOrderValue    = ("LineTotal", "mean"),
        TotalItems       = ("Quantity",  "sum"),
        DistinctProducts = ("StockCode", "nunique"),
        TenureDays       = ("Date",      lambda x: (x.max() - x.min()).days),
        AvgItemsPerOrder = ("Quantity",  "mean")).reset_index()
    return rfm
```

F.4 JWT authentication dependencies (app/auth_deps.py, app/routers/auth.py)

```
def _create_token(user_id, email):
    payload = {"sub": str(user_id), "email": email,
               "exp": datetime.now(timezone.utc) + TOKEN_EXPIRY}
    return jwt.encode(payload, JWT_SECRET, algorithm=JWT_ALGO)

def get_current_user_id(authorization = Header(None), token = Header(None)):
```

```
    raw = _strip_bearer(authorization) or (token.strip() if token else None)
    if not raw:
        raise HTTPException(401, "Missing Authorization token")
    try:
        payload = jwt.decode(raw, JWT_SECRET, algorithms=[JWT_ALGO])
    except jwt.ExpiredSignatureError:
        raise HTTPException(401, "Token expired")
    except jwt.InvalidTokenError:
        raise HTTPException(401, "Invalid token")
    return int(payload["sub"])

async def get_current_admin(user_id = Depends(get_current_user_id), conn = Depends(get_conn)):
    role = await conn.fetchval("SELECT role FROM users WHERE id = $1", user_id)
    if role != "admin":
        raise HTTPException(403, "Admin access required")
    return user_id
```

F.5 Behavioral event tracking (frontend/src/hooks/useAnalytics.js)

```
const sendEvent = (payload) => {
  const user = JSON.parse(localStorage.getItem("th_user"))
  fetch(`${API_BASE}/api/analytics/event`, {
    method: "POST", headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ session_id: sessionId, user_id: user?.id ?? null, ...payload }),
  }).catch(() => {})
}

// Safely find the nearest [data-track] ancestor
const closestTracked = (e) => {
  let node = e.target
  if (node && node.nodeType === 3) node = node.parentElement
  if (!node || typeof node.closest !== "function") return null
  return node.closest("[data-track]")
}

const onClick = (e) => {
  const el = closestTracked(e)
  if (!el) return
  sendEvent({ event_type: "click", element: el.dataset.track,
              value: el.dataset.trackValue || el.innerText?.slice(0, 60), page })
}
document.addEventListener("click", onClick)
document.addEventListener("mouseenter", onHover, true)
```

F.6 Chart data and image endpoint (app/routers/results.py, app/services/plots.py)

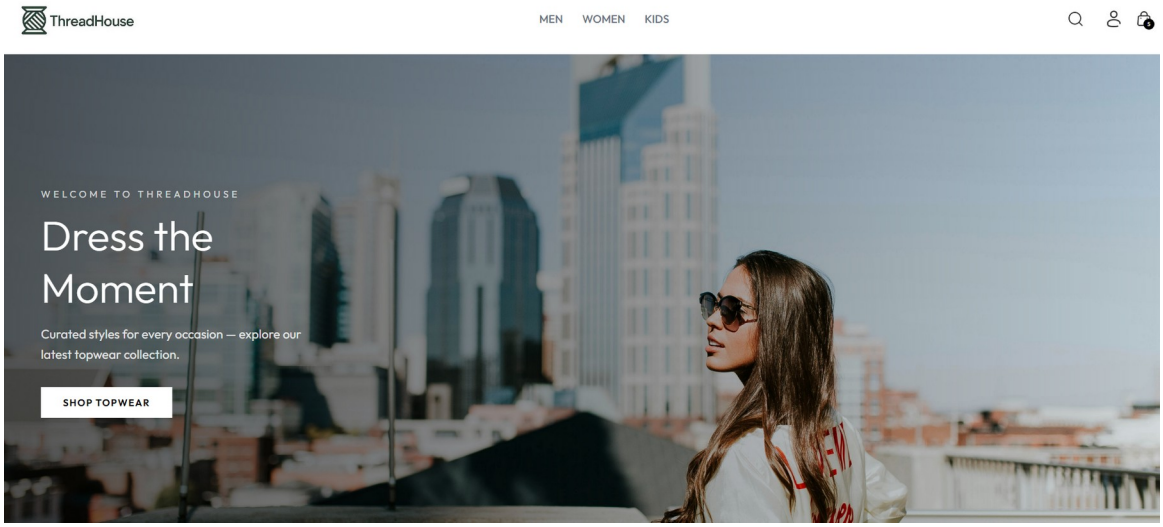
```
@router.get("/results/{job_id}/plots")
def get_plots(job_id, db = Depends(get_db)):
    profiles = db.query(CustomerProfile).filter(
        CustomerProfile.job_id == job_id).all()
    if not profiles:
        raise HTTPException(404, "No results found")
    data = build_plot_data(profiles)
    data["images"] = save_plots(job_id, profiles, data)   # writes PNG files
    return data

def build_plot_data(profiles):
    seg = defaultdict(int)
    for p in profiles:
        seg[p.segment or "Unknown"] += 1
    segment_distribution = [{"segment": s, "count": n}
        for s, n in sorted(seg.items(), key=lambda x: -x[1])]
```

```
rfm_scatter = [{"segment": p.segment, "recency": p.recency,
               "monetary": p.monetary} for p in profiles]
# ... clv_by_segment, hvr_potential, anomaly_breakdown, prob_alive ...
return {"segment_distribution": segment_distribution,
        "rfm_scatter": rfm_scatter}
```

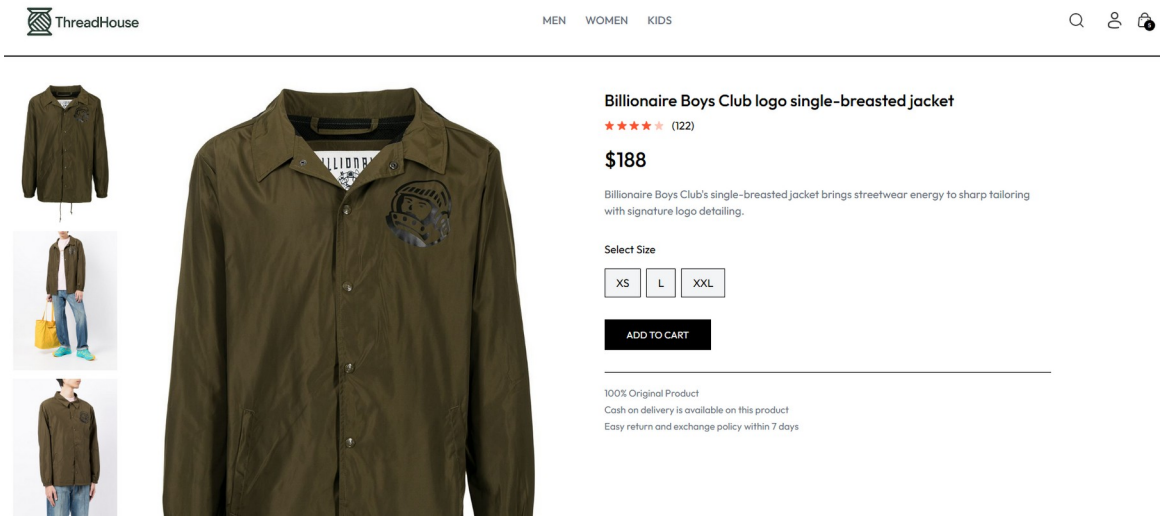
Appendix B: Application Screenshots

The following screenshots, captured from the running application, illustrate the main screens of the shop and the administrative panel.

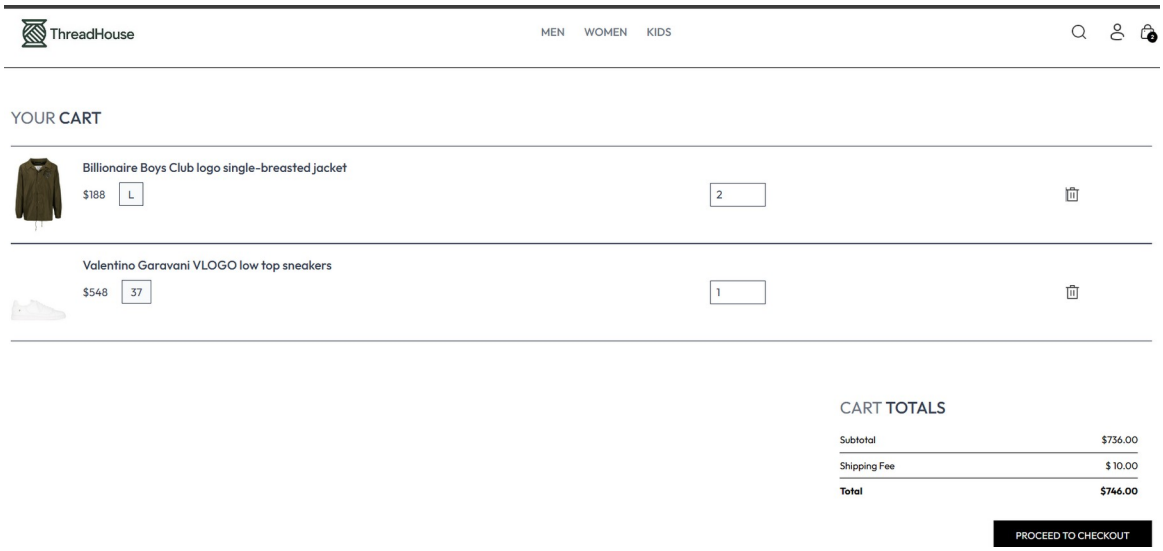
Shop Home Page



Product Detail Page

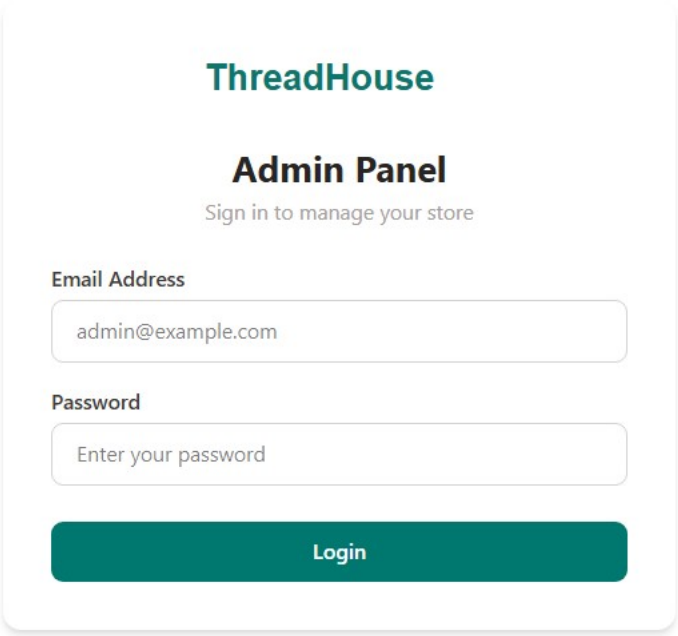


Shopping Cart and Checkout

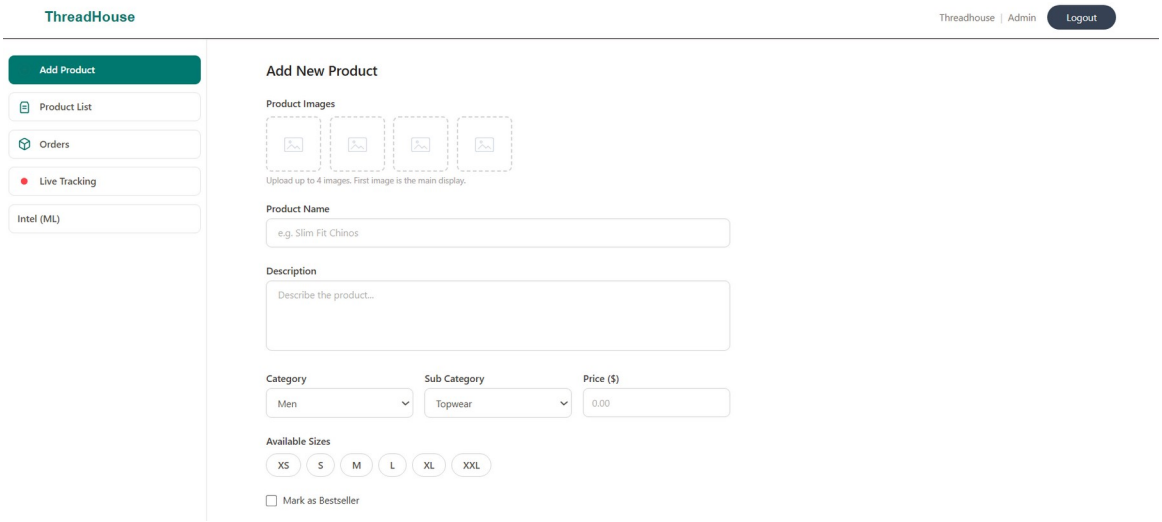


Customer Order History

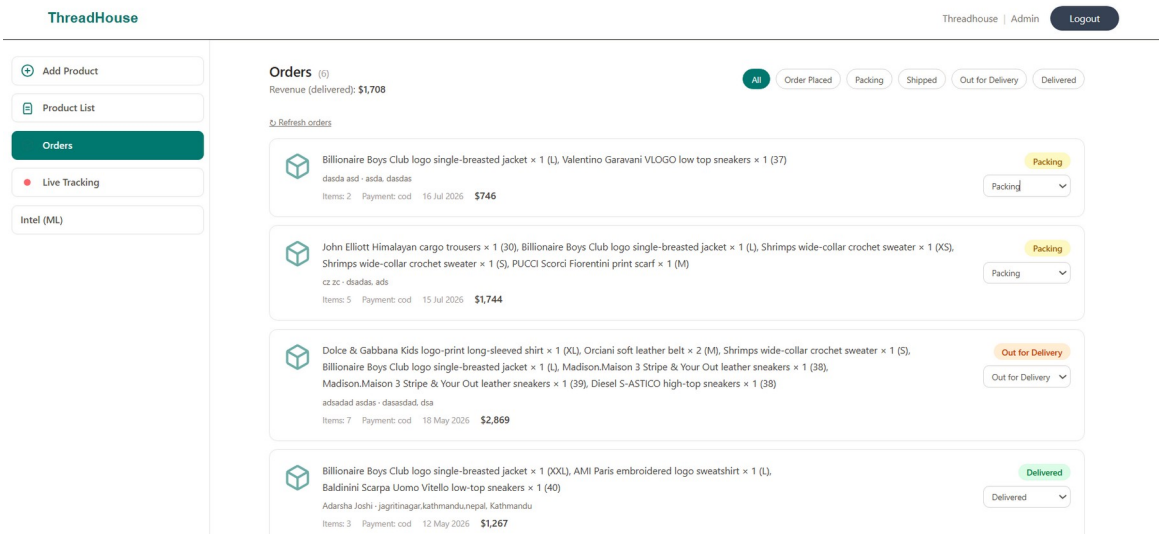
Admin Login



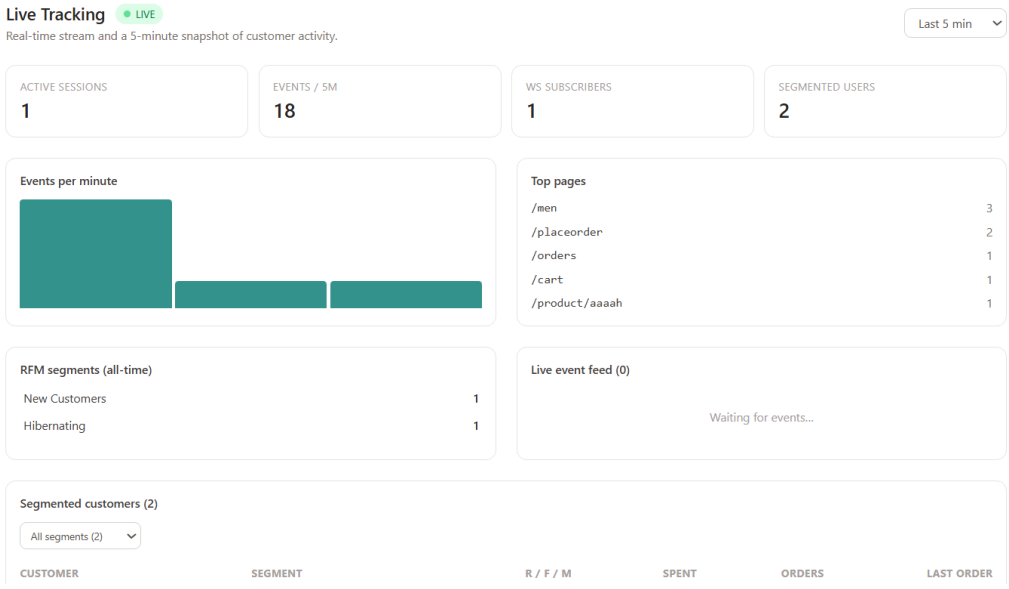
Admin Product Management



Admin Order Management

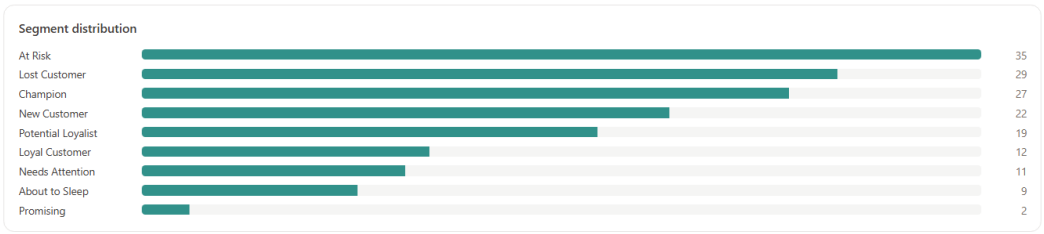


Live Tracking Dashboard



Intelligence Dashboard

CUSTOMERS	REVENUE	AVG CLV (12M)	ANOMALIES
166	$532,206.54	$3,400.38	9



Top customers by CLV

CUSTOMER	SEGMENT	CLV (12M)	SPEND	ALIVE
17159	Potential Loyalist	$183364.23	$101292.41	85%
17158	Needs Attention	$123889.28	$76637.07	76%
17156	Potential Loyalist	$53949.74	$23712.69	80%
17157	Needs Attention	$21208.99	$76132.01	70%
17002	Champion	$7758.48	$7931.05	96%
17001	Champion	$6957.12	$6789.80	98%
17021	Champion	$6669.54	$7340.81	97%
17009	Champion	$6285.77	$7622.98	96%
17012	Champion	$5846.53	$5526.50	97%
17015	Champion	$5832.91	$5800.43	98%

Insights